

DiStefano: Decentralized Infrastructure for Sharing Trusted Encrypted Facts and Nothing More

Private and Efficient Commitments for TLS-encrypted Data

Sofia Celi*, Alex Davidson†, Hamed Haddadi*¶, Gonçalo Pestana‡ and Joe Rowell§

* Brave Software, cherenkov@riseup.net, hamed@brave.com

† NOVA LINCS & DI, FCT, Universidade NOVA de Lisboa, a.davidson@fct.unl.pt

‡ Hashmatter, gpestana@hashmatter.com

§ Information Security Group, Royal Holloway, University of London, joe.rowell@rhul.ac.uk

¶ Imperial College London, h.haddadi@imperial.ac.uk

Abstract—We design **DiStefano**: an efficient, maliciously-secure framework for generating private commitments over TLS-encrypted web traffic, for verification by a designated third-party. **DiStefano** provides many improvements over previous TLS commitment systems, including: a modular protocol specific to TLS 1.3, support for arbitrary verifiable claims over encrypted data, client browsing history privacy amongst pre-approved TLS servers, and various optimisations to ensure fast online performance of the TLS 1.3 session. We build a permissive open-source implementation of **DiStefano** integrated into the BoringSSL cryptographic library (used by Chromium-based Internet browsers). We show that **DiStefano** is practical in both LAN and WAN settings for committing to facts in arbitrary TLS traffic, requiring < 1 s and ≤ 80 KiB to execute the complete online phase of the protocol.

1. Introduction

The Transport-Layer Security (TLS) protocol [1] provides encrypted and authenticated channels between clients and servers on the Internet. Such channels commonly transmit *trusted information* about users behind clients such as proofs of age [2], social security statuses [3], and accepted purchase information. While various applications would benefit from learning such data points, doing so represents an obvious privacy concern [4]–[9]. Exporting such information as anonymous credentials is non-trivial since the information resides in an encrypted and authenticated channel. Meanwhile, both legislation (such as GDPR [10]) and standards bodies (such as W3C [11]) have made usage of privacy-preserving data credentials a priority.

Designated-Commitment TLS (DCTLS) protocols (also known as *three-party handshake* protocols) provide modified TLS handshakes that allow exporting certain claims on the TLS channel to a designated *verifier*. The protocols perform handshakes that secret-share private session data amongst a client and a verifier, and compute the handshake and record-layer phases in two-party computation (2PC). Examples of DCTLS protocols include DECO [2],

TLSNotary/PageSigner [12]¹, TownCrier [13], Garble-then-Prove [14], and Janus [15]. Similar techniques are also used to produce zero-knowledge middleboxes [16] (for proving that client traffic adheres to corporate browsing policies, for example), and for devising multi-party TLS clients/servers. Prominent examples of the latter are Oblivious TLS [17] and MPCAuth [18].

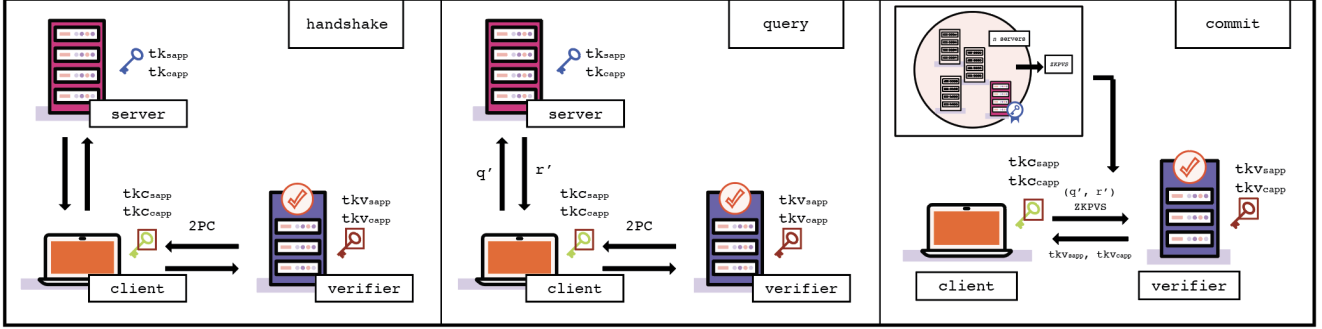
Unfortunately, while previous works claim practicality, all such DCTLS protocols appear insufficient for wide-scale usage. First, no protocol explicitly provides secure support for TLS 1.3. Support for TLS 1.3 surpassed that of 1.2 around December 2020 [19] and, according to Cloudflare Radar [20], TLS 1.3 now accounts for 63% of secure network traffic as opposed to 8.7% for TLS 1.2, so it is imperative that protocols support this version. When DCTLS protocols do support TLS 1.3, the security analysis is lacking and/or efficiency concerns that surround implementing TLS 1.3 ciphersuites and protocol steps in (maliciously-secure) 2PC are overlooked. Existing security arguments also lack in *agility*, meaning that they only apply for a static protocol, ciphersuite and 2PC primitives. This is a critical concern: primitives used by DECO have been already shown to be insecure [21], [22]. Client privacy is also neglected as the protocols reveal the server that clients communicate with to the verifier, revealing their browsing history. Second, from a deployability perspective, no fully-featured open-source implementation of a DCTLS protocol exists that achieves strong security guarantees, or much less one that interoperates with Internet browsing tools.

Our work. We design **DiStefano** (Fig. 1), a DCTLS protocol that securely generates private commitments over TLS 1.3 data. Security is proven using a novel standalone model that permits cryptographic agility by allowing to swap various schemes depending on the desired ciphersuite. **DiStefano** is provided as a permissive open-source implementation² integrated into the widely-used BoringSSL

1. We refer exclusively here to original PageSigner as TLSNotary does not appear to have a fixed cryptographic design.

2. <https://github.com/brave-experiments/DiStefano>

Figure 1. An overview of the DiStefano protocol. In the **handshake** and **query** phases, the client performs the TLS 1.3 handshake and record-layer phases in conjunction with the verifier using 2PC to secret-share traffic keys and other session data for establishing a secure session with the server (secret-shared keys are represented with a square over the key). In the **commitment** phase, the client authenticates the server to the verifier using a zero-knowledge proof of valid TLS signatures (denoted by ZKPVS, see Section B.3), and commits to some encrypted session data, before receiving the verifier's secret TLS session shares.



library,³ where 2PC functionality is provided by emp [23]. With respect to the client's privacy, DiStefano supports zero-knowledge authentication amongst N verifier-approved TLS servers by using a zero-knowledge proof of valid signatures. Finally, the commitments generated by DiStefano can be used to produce any type of verifiable private claim, either non-interactively using zero-knowledge proofs or interactively using 2PC. Note that, in this work, we value the building of a modular framework for solving the core commitment functionality, and leave the implementation of the subsequent proving stage up to the implementer, as this can differ significantly depending on the application (see Section 4.4). To ensure high performance, a number of optimisations were made to the cryptographic functionality and software implementation of DiStefano. We show that the online portions of the handshake and record-layer phases can be executed in 500 ms and ~ 750 ms in a LAN setting, and with 5 KiB and around 4 KiB of bandwidth, respectively, for 2 KiB of TLS communication. In a WAN setting, there are modest increases in timing that are largely explained by the increase in latency. All in all, the online costs of DiStefano fall way under a second, which is far below standard TLS handshake timeout times of 10-20 seconds [24].

Formal contributions. Our formal contributions are:

- A private Designated-Commitment TLS 1.3 (DCTLS) protocol, DiStefano (Section 4), with a modular, standalone security framework that guarantees security in the presence of malicious adversaries (Section 6).
- Novel optimisations that allow running secure 2PC TLS 1.3 clients with higher efficiency (Section 5).
- An open-source, Chromium-compliant implementation integrated into BoringSSL.
- Experimental analysis showing that DiStefano is practical (in LAN/WAN settings) for committing to various sizes of Internet traffic (Section 7).

3. This library is used by most Chromium-based Internet browsers, that make up a dominant share of all browser usage.

2. Background

2.1. General Notation

Vectors are denoted by lower-case bold letters. We use $\text{len}(s)$ to denote the length of $s \in \{0, 1\}^*$. The symbol $[m]$ indicates the set $\{1, 2, \dots, m\}$. We write $a \leftarrow b$ to assign the value of b to a , and $a \leftarrow \$S$ to assign a uniformly sampled element from the set S . λ denotes the security parameter.

We denote a finite field of characteristic q as $\mathbb{F}_q$ and the m -dimensional vector space over $\mathbb{F}_q$ as $\mathbb{F}_q^m$. We are primarily concerned with the smallest field, $\mathbb{F}_2$, where the additive operation on $a, b \in \mathbb{F}_2$ is simply an *exclusive-or* operation, $a \oplus b$, with multiplication corresponding to the AND operation. We extend this notation to refer to operations on m -dimensional vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}_2^m$, writing $\mathbf{a} \oplus \mathbf{b}$ and $\mathbf{a} \cdot \mathbf{b}$ to refer to addition and multiplication, respectively. Note that while addition in $\mathbb{F}_2^m$ is simply m XOR operations, multiplication over $\mathbb{F}_2^m$ requires extra logic compared to multiplications over $\mathbb{F}_2$. We write elliptic curves with a generator G over $\mathbb{F}_q$ as $EC(\mathbb{F}_q)$.

For a security game **Game** used by a cryptographic scheme Δ , we denote the advantage of an algorithm $\mathcal{A}$ in Δ by $\text{Adv}_{\mathcal{A}, \Delta}^{\text{game}}(\lambda)$, where:

$$\text{Adv}_{\mathcal{A}, \Delta}^{\text{game}}(\lambda) = \Pr[\mathcal{A} \text{ succeeds}] - \Pr[\mathcal{A} \text{ fails}]. \quad (1)$$

We say that Δ is secure with respect to **Game**, iff $\text{Adv}_{\mathcal{A}, \Delta}^{\text{game}}(\lambda) \leq \text{negl}(\lambda)$, for some negligible function $\text{negl}(\lambda)$ and security parameter λ . In the following, we will use certain acronyms: we provide a glossary of them in Appendix L.

2.2. Background on DCTLS Protocols

Designated-Commitment (DCTLS) TLS protocols allow a client (C) to generate commitments to TLS session data communicated with a server (S) that can be sent to a

designated third-party verifier ($\mathcal{V}$). They consist of the following phases (which are described in Appendix C): a ($\mathcal{V}$ -assisted) handshake phase, a ($\mathcal{V}$ -assisted) query execution phase, and a commitment phase. Previous work, such as in DECO and tools like PageSigner, provide explicit attestation functionality for proving facts about the committed TLS session (using zero-knowledge proofs). Note that, without such commitments, proving statements that use TLS data as sources of truth must assume either a trustworthy client, or $\mathcal{C}$ must allow $\mathcal{V}$ to read their TLS traffic in the clear.

DCTLS over TLS 1.3. Previous DCTLS protocols focused on TLS 1.2, with an informal (and mostly incomplete) extension to TLS 1.3. Recall that TLS 1.3 emerged in response to dissatisfaction with the outdated design of the TLS 1.2 handshake, its two-round-trip overhead, and the increasing number of practical attacks [25]–[28]. The necessary changes introduced by TLS 1.3 to improve performance and deployability are significant stumbling blocks for applying previous DCTLS protocols directly. Thus, in this work, we focus on TLS 1.3, and highlight explicit changes to DCTLS protocols that are required for handling the substantial protocol-level differences. We provide an overview of the standard TLS 1.3 handshake, and its standard notation defined in [29], in Appendix A.

Description of DCTLS phases. In Fig. 1, we summarise the stages of DCTLS for establishing commitments to TLS 1.3 encrypted traffic between $\mathcal{C}$ and $\mathcal{S}$ to be sent to a designated $\mathcal{V}$. In the following, we describe how the different stages of the protocol function in relation to the various stages of the TLS 1.3 protocol [1]. The following is an informal description of TLS 1.3 (1-RTT with certificate-based authentication) when extended to support DCTLS-like protocols.

Handshake phase. In this phase, $\mathcal{S}$ learns the same secret session parameters as in standard TLS 1.3, while $\mathcal{C}$ and $\mathcal{V}$ learn shares of the session parameters that only $\mathcal{C}$ would normally learn. This requires $\mathcal{C}$ and $\mathcal{V}$ to engage in the core TLS 1.3 protocol using a series of 2PC functionalities.

We focus on the default mode for establishing a secure TLS 1.3 session using (EC)DH ciphersuites, and certificate-based authentication between $\mathcal{C}$ and $\mathcal{S}$. In this mode, the handshake starts with $\mathcal{C}$ sending a `ClientHello` (CH) message to $\mathcal{S}$. This message advertises the supported (EC)DH groups and the ephemeral (EC)DH keyshares specified in the `supported_groups` and `key_shares` extensions, respectively. The CH message also advertises the signature algorithms supported. It also contains a nonce and a list of supported symmetric-key algorithms (ciphersuites). Note that for DCTLS protocols, the ephemeral keyshares $Z \in EC(\mathbb{F}_c)$ are generated as a combination of additive shares ($z_X \leftarrow \mathbb{F}_c, Z_X = z_X \cdot G$) for $X \in \{\mathcal{C}, \mathcal{V}\}$, where $Z = Z_C + Z_V \in EC(\mathbb{F}_c)$.

$\mathcal{S}$ processes the CH message and chooses cryptographic parameters for the session. If (EC)DH key exchange is used, $\mathcal{S}$ sends a `ServerHello` (SH) message containing a `key_share` extension with their (EC)DH key, corresponding to one of the `key_shares` advertised by $\mathcal{C}$. The SH message also contains a $\mathcal{S}$ -generated nonce and the

ciphersuite chosen. An ephemeral shared secret is computed at both ends, which requires $\mathcal{C}$ and $\mathcal{V}$ to engage in 2PC computation. Afterwards, all subsequent handshake messages are encrypted using keys derived from this secret. Once this derivation is performed, $\mathcal{V}$'s keys can be revealed to $\mathcal{C}$ to perform local encryption/decryption of handshake messages, as these keys are considered independent from the eventual session secret derived at the end of the handshake [29].

The server $\mathcal{S}$ then sends a certificate chain (in the `ServerCertificate` message -SCRT-), and a message that contains a proof that they possess the private key corresponding to the public key advertised in the leaf certificate. This proof is a signature over the handshake transcript and it is sent in the `ServerCertificateVerify` (SCV) message. $\mathcal{S}$ also sends the `ServerFinished` (SF) message that provides integrity of the handshake up to this point. It contains a message authentication code (MAC) over the entire transcript, providing key confirmation and binding $\mathcal{S}$'s identity to any computed keys. Optionally, $\mathcal{S}$ can send a `CertificateRequest` (CR) message, prior the SCRT message, requesting a certificate from $\mathcal{C}$.

At this point, $\mathcal{S}$ can immediately send application data to the unauthenticated $\mathcal{C}$. Upon receiving $\mathcal{S}$'s messages, $\mathcal{C}$ verifies the signature of the SCV message and the MAC of SF. If requested, $\mathcal{C}$ responds with their own authentication messages, `ClientCertificate` and `ClientCertificateVerify`, to achieve mutual authentication. Finally, $\mathcal{C}$ must confirm their view of the handshake by sending a MAC over the handshake transcript in the `ClientFinished` (CF) message. The MAC generation must also be computed in 2PC.

Now, the handshake is completed, and $\mathcal{C}$ and $\mathcal{S}$ can derive the key material required by the subsequent *record layer* to exchange authenticated and encrypted application data. This derivation is performed in 2PC, and $\mathcal{C}$ and $\mathcal{V}$ both hold shares of all the secret parameters needed to encrypt traffic using the specified encryption ciphersuite. In this work, we specifically target AES-GCM, since over 90% of TLS 1.3 traffic uses this ciphersuite [30].

Record Layer (Query Execution) phase. $\mathcal{C}$ sends a query q (in encrypted form $\hat{q}$) to $\mathcal{S}$ with help from $\mathcal{V}$. Specifically, since the session keys are secret-shared, $\mathcal{C}$ and $\mathcal{V}$ jointly compute the encryptions of these queries in 2PC. Encrypted responses, $\hat{r}$, can then be decrypted using a similar procedure to reveal $\mathcal{S}$'s response r to $\mathcal{C}$. This is important for running tools in a browser, or any multi-round protocol, where subsequent queries depend on previous responses.

Commitment phase. After querying $\mathcal{S}$ and receiving a response r , $\mathcal{C}$ commits to the session by forwarding the ciphertexts to $\mathcal{V}$, and receives $\mathcal{V}$'s session key shares in exchange. Hence, $\mathcal{C}$ can verify the integrity of r , and later prove statements about it. The fact that $\mathcal{C}$ sends commitments before they receive $\mathcal{V}$'s shares means that $\mathcal{V}$ can trust subsequent attestations over the commitments.

Limitations of approaches. Existing DCTLS schemes have serious security, performance, and deployability limitations. They either only work with old/deprecated TLS versions

(1.2 and under) and offer no privacy from the oracle (Page-Signer [31]), or rely on trusted hardware (Town Crier [13]) against which various attacks exist [32]. Another class of oracle schemes assumes cooperation from $\mathcal{S}$ by installing TLS extensions [33], or by changing application-layer logic [34]. These approaches suffer from two fundamental problems: they break legacy compatibility, causing a significant barrier to wide adoption; and only provide conditional exportability as $\mathcal{S}$ has the sole discretion to determine which data can be exported, and can censor export attempts. While DECO [2] promises to solve these problems, its non-modular security design, combining all of the individual phases of the protocol, makes it impossible to swap individual pieces of functionality (without rewriting the entire security proof). These limitations have the following repercussions.

Security. Some primitives used by DECO have since been shown to be insecure [21], [22], and the security proof only targets TLS 1.2. General guidance is offered for handling TLS 1.3, but is not formally specified. More worryingly, the security argument is all-encompassing, proving security for a combined session involving the TLS handshake, subsequent record protocol, and the zero-knowledge attestation layer (this is common in other constructions too [13], [18]). This significantly harms *cryptographic agility*, since *any* change to the utilised primitives, protocol, or ciphersuites theoretically dictates that an entirely new proof be written. Lack of cryptographic agility is a significant source of cryptographic vulnerabilities in real-world systems [35].

Privacy. Explicit authentication of $\mathcal{S}$ to $\mathcal{V}$ during the handshake is mandated due to the non-modular security proof, which is harmful for $\mathcal{C}$'s browsing privacy. This is somewhat necessitated by TLS 1.2's handshake flow, where the identity of $\mathcal{S}$ is sent in the clear in the first message they sent. By contrast, TLS 1.3 rearranges the handshake flow slightly, and the certificate is only exchanged in an encrypted form on later messages. Because of this, DiStefano achieves modular privacy properties and retains the privacy of $\mathcal{C}$ (we never disclose the identity $\mathcal{S}$ to $\mathcal{V}$, as this will reveal the client's browsing history, for example).

Performance. Certain underlying cryptographic tools (such as oblivious transfer protocols) have seen remarkable improvements following DECO's publication [36], [37]. However, certain parts of the transformations needed to handle the AES-GCM ciphersuite detailed by DECO are underspecified, and naively lead to high costs during 2PC execution.

Deployability. Recent DCTLS protocols [14], [15], [38], [39] are either aimed entirely at TLS 1.2 [14], are entirely theoretical [38], or use semi-honest 2PC to achieve reasonable performance [15], [39]. We note that the use of semi-honest 2PC must be applied carefully to guarantee overall malicious security of the protocol, and (to the best of our knowledge) no TLS 1.3 attestation mechanism has yet been proposed that completely satisfies this. In fact we discuss in Appendix K that using semi-honest 2PC primitives may lead to potential attacks. Moreover, even when semi-honest 2PC is used, performance is lacking and public implementations are rare. One recent example is the recently proposed

Janus protocol [15], which is accompanied by a reference implementation, but without integration into existing TLS libraries. Janus achieves a reported handshake time of around 0.51 s in a LAN setting with around 113 KiB of handshake traffic. By contrast, DiStefano achieves malicious security guarantees, with around 0.1 ms of online time, and exchanging much less data: around 28 KiB for the online phase of the handshake. Thus, we conclude that using malicious 2PC is not a bottleneck for current protocols.

2.3. Overview of DiStefano

Due to the limitations of the previous DCTLS protocols, we aim to build a protocol that works for TLS 1.3, improves privacy guarantees for $\mathcal{C}$, does not require specific hardware or extensions, and can be easily integrated into common applications. Overall, DiStefano achieves the following.

- The creation of a maliciously-secure framework that generates binding and hiding commitments over data communicated during TLS 1.3 sessions.
- Cryptographic optimisations that ensure practical running costs, and experimental analysis showing that DiStefano is ready for real workflows.
- A publicly-available implementation integrated into the TLS library that browsers use, with no need for specialized hardware or installing extra extensions.

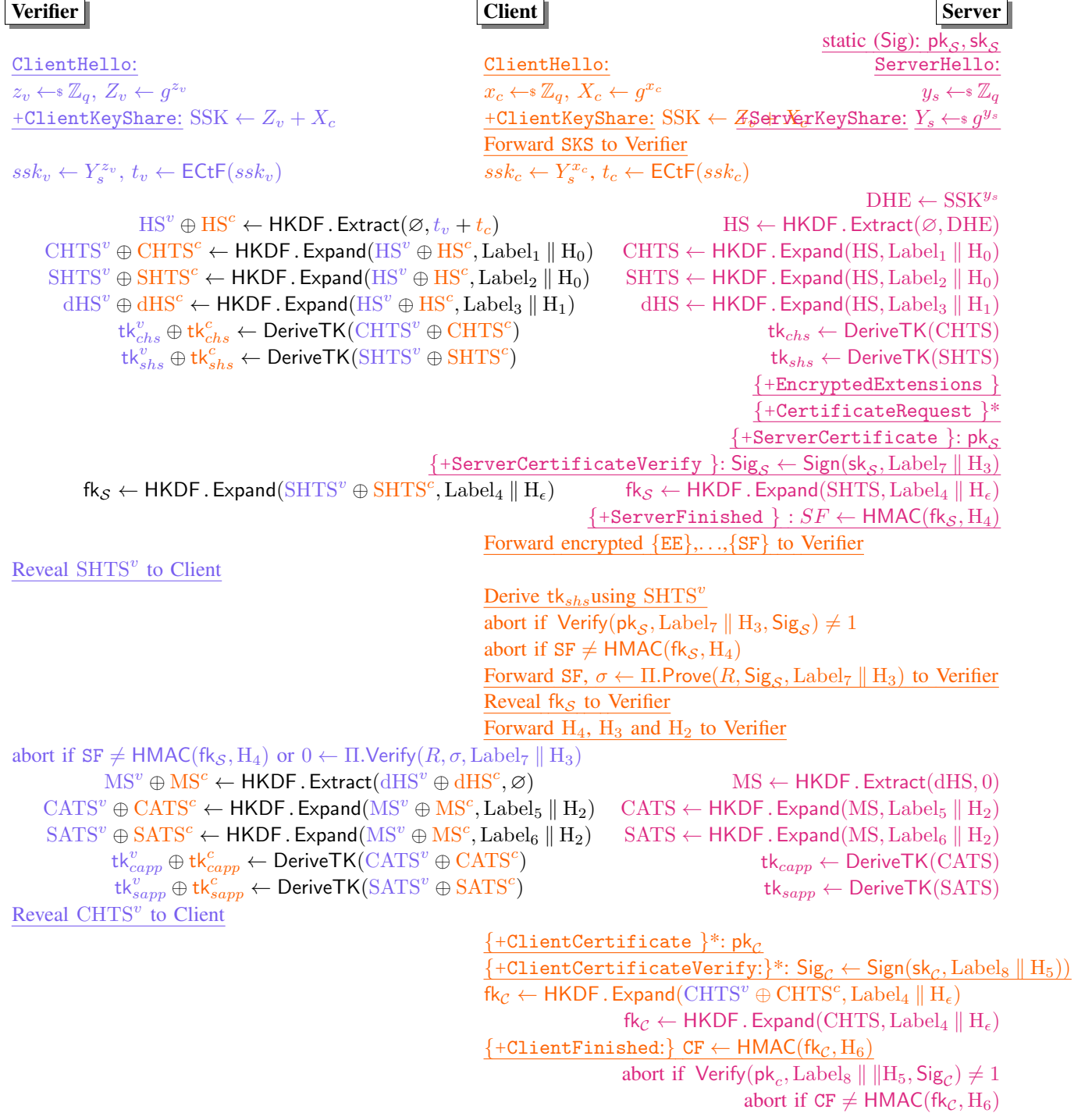
We believe that DiStefano is an essential step-forward for showing that DCTLS *can* be implemented in practice.

Overview of required optimisations. Our implementation of DiStefano requires several optimisations to achieve its performance. We reduce the number of rounds required to derive AES-GCM secret shares (cf. Section 5) by a factor of around 500 compared to prior art (PageSigner), and reduce the required bandwidth by around a factor of 3. Moreover, we carefully combine multiple sub-circuits used in the TLS handshake to reduce the number of re-computed secrets and circuit invocations (cf. Section 4.1). We emphasise that a significant portion of our engineering effort was dedicated to fine-tuning at a low level, and we view this as a valuable contribution in its own regard: we aspire that this facilitates seamless adaptation of our code by future researchers.

3. Secure Multi-Party Computation

Two-party secure computation (2PC) protocols allow parties p_1 and p_2 to jointly compute generic functions $f(s_1, s_2)$ over their private inputs s_1 and s_2 . The security of the protocols ensures that nothing of each input is revealed to the other party, except for what f naturally reveals [40]. There are two common approaches for 2PC protocols. *Garbled circuits protocols* [41], [42] encode f as a boolean circuit and evaluate an encrypted variant of the circuit across two parties. *Threshold secret-sharing* protocols (e.g. SPDZ [43], [44], or MASCOT [45]), typically operate by first producing some random multiplicative triples (referred to as *Beaver triples* [46]) before additively sharing secret

Figure 2. The DiStefano 1-RTT handshake protocol. Shorthands correspond to those defined in [29]. **Purple** represents messages sent or calculated by $\mathcal{V}$, **orange** by the client, **pink** by the server, and **black** for 2PC calculations between the client and verifier. Messages with an asterisk (*) are optional, and those within braces ({}) are encrypted.



inputs with some extra information. Garbled circuit protocols are particularly well-suited to secure evaluation of binary circuits, such as AES or SHA-256. The cost of a garbled circuit is normally evaluated in terms of the number

of AND gates due to the *Free-XOR* optimisation [47]. In contrast, threshold secret-sharing schemes are typically well-suited for computing arithmetic operations, such as modular exponentiation. We calculate their cost in terms of their

number of rounds and bandwidth requirements.

MPC primitives. We use both types of 2PC protocols: maliciously-secure authenticated garbling implementation provided by `emp` [48] for binary operations, and we base 2PC arithmetic operations on *oblivious transfer* (OT).

Definition 1 (Oblivious Transfer (OT)). *An oblivious transfer scheme, OT, consists of the following algorithms:*

- $\text{OT.Gen}(1^\lambda)$: *outputs any key material.*
- $\text{OT.Exec}(m_0, m_1, b)$: *The sender P_1 inputs messages m_0 and m_1 , and the receiver P_2 inputs a bit $b \in \{0, 1\}$. The receiver P_2 learns m_b , while P_1 learns nothing.*

We realise the OT functionality via the actively secure IKNP [49], [50] extension and the Ferret [36] OT scheme. Both rely on the security of information theoretic MACs, the *learning parity with noise* (LPN) assumption, and on randomness assumptions about hash functions, see [51].

Using OT as a building block, we realise the remaining 2PC functionality needed by using *multiplicative-to-additive* (MtA) secret sharing schemes.

Definition 2 (MtA). *An MtA scheme, MtA, consists of the following algorithms:*

- $\text{MtA.Gen}(1^\lambda)$: *outputs any needed key material.*
- $\text{MtA.Mul}(\alpha, \beta)$: *each P_i supplies a_i , learning as output b_i , such that $\sum b_i = \Pi_i a_i$.*

For malicious security, we expand this definition with an additional algorithm, $\text{MtA.Check}(a_1, \dots, b_1, \dots)$, to check share consistency. Existing works [2], [12], [15] realise MtA with an approach [52] based on Paillier encryption [53]. We deviate from this approach to improve efficiency [54, §5], and to mitigate the need for range proofs [21], [22] (necessary for achieving malicious security). We realise the MtA functionality using the schemes introduced in [55] and [56], [57] for rings of characteristic > 2 and 2, respectively. The schemes require OT functionality and are instantiated with 128-bit statistical and computational security. We note that whilst the security of [56], [57] reduces directly to an NP-hard encoding problem [58], to the best of our knowledge, there is no computational hardness proof for [55].

ECtF. During the *Key Exchange* phase of the handshake of DCTLS, both $\mathcal{V}$ and $\mathcal{C}$ hold additive shares Z_v and Z_c of a shared ECDH key $(x, y) = \text{DHE}$. Given that all key derivation operations are carried out on the x co-ordinate of Z , we use the *elliptic curve to field* (ECtF) functionality [2] to produce additive shares t_v and t_c of the x coordinate, which is an element in $\mathbb{F}_q$. Using these shares as inputs to the subsequent 2PC operations to derive the handshake secrets allows running all computation in a binary circuit, which results in a substantial performance improvement compared with attempting to combine arithmetic and binary approaches in a garbled circuit. We stress that use of the ECtF functionality improves performance: we estimate that computing just the x co-ordinate of $Z_v + Z_c$ in a garbled circuit would be more expensive than deriving all TLS session secrets, requiring around 1.7M AND gates for an elliptic curve over a field with a 256-bit prime. From a

security perspective, we remark that the security of the ECtF functionality reduces to the security of the underlying secure multiplication protocol. We achieve malicious security by instantiating the multiplication with a maliciously-secure MtA scheme.

4. DiStefano Protocol

In this section, we fully describe each of the phases of the DiStefano protocol (formal ideal functionalities are given in Appendix C). A diagram of the full protocol is found in Fig. 2, and, for comparison, we provide a diagram of TLS 1.3 and a summary of the shorthands used (from [29]) in Appendix A. The security analysis is found in Section 6 and Appendix E.

4.1. Handshake Phase: HSP

We use the similar overarching mechanism for the handshake phase as described in Section 2.2, but focused exclusively on TLS 1.3 with AES-GCM as the AEAD scheme (Section B.2), using ECDH for the shared key generation, and using ECDSA certificates. The 2PC ideal functionalities that we use are defined in Algorithms 3 to 5 (Section E.1). The protocol can be adapted to work with any other TLS 1.3-compliant ciphersuites that are compatible with 2PC.

At a high-level, we adapt the TLS 1.3 handshake by treating $\mathcal{C}$ and $\mathcal{V}$ as a *single* TLS client from the perspective of $\mathcal{S}$. For this, we reverse the “traditional” flow of the TLS 1.3 handshake by having $\mathcal{C}$ and $\mathcal{V}$ each prepare an additively shared ephemeral key share SSK , as seen in Fig. 2. This can be computed without 2PC.

$\mathcal{C}$ then sends the CH and the CKS messages, advertising SSK as part of the `key_shares` extension. $\mathcal{S}$ then processes these messages and, in turn, sends a SH message back to $\mathcal{C}$ containing a freshly sampled ECDH `key_share` Z_s . At this stage, $\mathcal{S}$ computes the shared ECDH key as $\text{DHE} = x_s \cdot SSK$ and continues to derive all traffic secrets (i.e. CHTS, SHTS, tk_{shs} , tk_{chs}). Once $\mathcal{C}$ and $\mathcal{V}$ receive the SH message, they derive additive shares of the shared ECDH key as $E = x_c \cdot Y_s + x_v \cdot Y_s$. As TLS 1.3 key derivation operates on the x coordinate of the shared key, $\mathcal{C}$ and $\mathcal{V}$ convert their additive shares of $E = (E_x, E_y)$ into additive shares $E_x = t_c + t_v$ by running the ECtF functionality. With E_x computed, $\mathcal{C}$ and $\mathcal{V}$ proceed to run the handshake key derivation circuit in 2PC, with each party learning shares $\text{HS}^v \oplus \text{HS}^c = \text{HKDF.Extract}(\emptyset, t_v + t_c)$. In practice, this process is carried out inside a garbled circuit that produces shares of CHTS, SHTS and dHS, as well as the SF message key fk_S . This key is provided to both $\mathcal{C}$ and $\mathcal{V}$. This circuit comprises of around 800K AND gates, which is similar to DECO’s circuit size for TLS 1.2. We delay the derivation of the traffic keys as it provides authenticity guarantees to $\mathcal{V}$.

Authentication phase. $\mathcal{S}$ sends the CR (if wanted), SCRT, SCV and SF messages. The SF message is computed by first

deriving a finished key fk_S from SHTS and then computing a MAC tag over a hash of all the previous handshake messages. At this point, S is able to compute the client application traffic secret, CATS, and the server application traffic secret, SATS. S can also start sending encrypted application data (under tk_{sapp}) while waiting for the final flight of C messages.

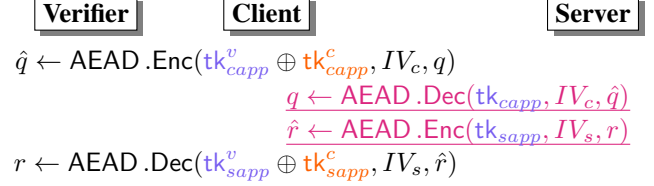
C receives the encrypted messages from S and, in turn, forwards them (encrypted) to V alongside a commitment to their share of CHTS/SHTS. This commitment is necessary to make AES-GCM act as a committing cipher from the perspective of V , which allows V to disclose their shares of CHTS and SHTS to C without compromising authenticity guarantees. As C now knows the entirety of CHTS and SHTS, they are able to locally derive the handshake keys tk_{chs} and tk_{shs} , allowing them to check S 's certificate and SF messages without the involvement of V . Moreover, as C now knows tk_{chs} they are also able to respond to the CR if one exists. C then forwards an authentic copy of the hashes H_2 , H_3 , and H_4 to V , allowing them to check the SF message as is done in TLS 1.3 (note that V needs to compute the hashes themselves to avoid replay attacks [59]). If privacy is wanted, notice that C does not forward the decrypted SCRT message to V , as this message reveals the identity of S . Let R be a set of TLS certificates, corresponding to a set of pre-approved TLS servers (S s). At this point, C can (optionally) send to V a zero-knowledge proof ($\sigma \leftarrow \Pi.\text{Prove}(R, \text{Sig}, \text{Label}_7 \parallel H_3)$) of a valid TLS signature (ZKPVS), Sig , as long as $S \in R^4$. V can then check the validity of the produced ZKPVS proof (if present) by checking that $1 \leftarrow \Pi.\text{Verify}(R, \sigma, \text{Label}_7 \parallel H_3)$.⁵ Similarly, C can selectively reveal the blocks containing the SF message, allowing V to validate the SF (note that fk_S was computed in 2PC, but it is revealed after the commitment). If the ZKPVS is used, V has to additionally check in ZKP that the forwarded hashes have been computed correctly to avoid replay attacks [59]. Finally, C and V derive the shares of the traffic secrets MS, CATS, SATS and the traffic keys tk_{sapp} , tk_{capp} in 2PC. In practice, we instantiate this derivation as a garbled circuit that contains around 700K AND gates. Note that this circuit cannot cheaply be combined with the handshake secret derivation circuit, as deriving the traffic keys requires a hash of the unencrypted handshake transcript. This would require decrypting and hashing large messages inside a garbled circuit, which is expensive.

4.2. Query Execution Phase: QP

Once HSP has completed, C and V move into the query phase (Fig. 3). For simplicity, we describe this portion of the protocol in terms of a single round of queries, before extending the phase to multiple rounds.

During the query phase of the protocol, C produces a series of queries $q = q_1, \dots, q_n$ and jointly encrypts these

Figure 3. The DiStefano query phase. Purple represents messages sent or calculated by V , orange by C , pink by S , and black for 2PC between C and V .



with V , with both parties learning a vector of ciphertexts $\hat{q}$ as output. Then, C forwards $\hat{q}$ to S , receiving an encrypted response $\hat{r}$ in exchange. At this stage of the protocol, C forwards $\hat{r}$ to V so that both parties may verify the tags on $\hat{r}$: both parties learn a single bit indicating if the tag check passed or not. In practice, we instantiate this portion of the protocol using the AES-GCM approach described in Section 5. There is no explicit dependence on AES-GCM: any AEAD cipher supported by TLS 1.3 will suffice. We highlight this and provide the general security formalisation of the phase in Section 6.

Extending the query phase to multiple rounds is straightforward using AES-GCM. We discuss the details of committing to ciphertexts in Section 5.1, but the main idea is that, as each ciphertext block q_i is encrypted with a unique key $e_i = \text{AES}.\text{Enc}(k, IV + i)$, C and V can arbitrarily reveal their shares of e_i at any stage of the query phase, provided an appropriate commitment has been made beforehand. As these key shares are ephemeral, revealing them does not compromise the shares derived during the HSP. The security of this approach directly reduces to the difficulty of recovering an AES key from many known plaintext/ciphertext pairs. This permits many useful applications, as C and V can now nest commitment rounds inside of the query phase.

4.3. Commitment Phase: CP

The objectives of the commitment phase (Fig. 4) are: i. to assure V of the authenticity of S (as belonging to the pre-approved set R) without revealing the exact server C communicated with; and ii. to allow C to learn secrets held by V only after producing binding commitments to a specific portion of the TLS session with S .⁶

To validate the authenticity of the server, V verifies a proof (with a ZKPVS scheme) of the TLS server that they communicated with as one of N servers from which V accepts attestations.⁷ After verification, C can now commit to and reveal information about the application traffic they witness. For this, first, we define a commitment scheme (Γ) that can be implicitly constructed using the outputs of QP with the following algorithms.

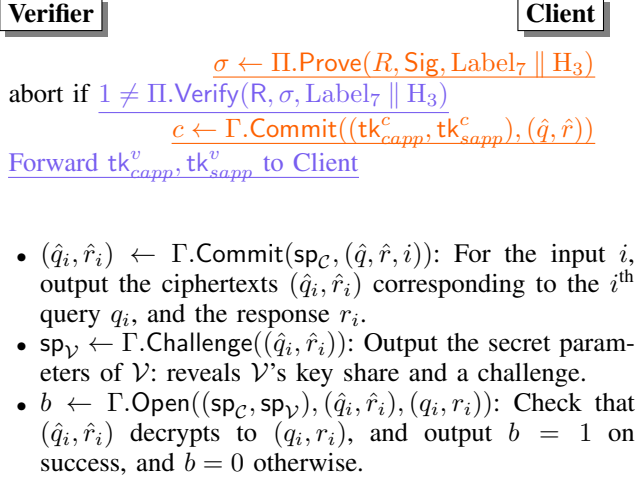
4. We detail a formalisation of ZKPVS schemes in Section B.3. Practical variants of such schemes exist for ECDSA signatures [60]–[62].

5. Sending and verifying the proof can be alternatively performed later in the DCTLS protocol, without compromising security.

6. Note this applies for both the handshake and record phases.

7. This could be performed during the handshake phase. For performance reasons, it is preferable to do so in the commitment phase, when online communication is no longer constrained by potential handshake time-outs.

Figure 4. The DiStefano commitment phase with a commitment scheme, Γ , and a ZKPVS scheme, Π for TLS signatures under a set R of pre-approved servers. **Purple** represents messages sent or calculated by $\mathcal{V}$, and **orange** by $\mathcal{C}$.



Note that the client simply commits to encrypted traffic exchanged during QP (using 2PC to encrypt and decrypt the traffic). When it comes to opening the encrypted traffic, the protocol requires $\mathcal{V}$ to reveal their key secret share, so that $\mathcal{C}$ can decrypt and then reveal the plaintext values. We give a concrete construction of Γ that is perfectly hiding and computationally binding, based on AES-GCM, in Section D.2.

4.4. Subsequent Phases

It is important to note that in the real DiStefano protocol, $\mathcal{C}$ does not send any unencrypted values to $\mathcal{V}$. Instead, both parties should execute a protocol that proves certain facts about the DCTLS commitments, without revealing anything else. This could be done using zero-knowledge proofs, selective opening strategies (as is used in DECO), or subsequent 2PC. The formal commitment opening process that we described previously can be used for this, since $\mathcal{C}$ can now use the combined secret parameters $(\text{sp}_C, \text{sp}_V)$ to prove any statement about the commitment $(\hat{q}_i, \hat{r}_i)$. Note that the proving process can inadvertently leak the identity of $\mathcal{S}$ (invalidating the ZKPVS proof) if a certain data is assumed from $\mathcal{S}$'s traffic. See Appendix G for a wider discussion.

5. Adapting AES-GCM For 2PC

AES-GCM is an authenticated encryption with associated data (AEAD) cipher that features prominently inside TLS implementations, with some works reporting that over 90% of all TLS1.3 traffic is encrypted using AES-GCM [30]. We use this algorithm for both encrypting the corresponding handshake messages and any application traffic. Here we describe how to commit to decryptions of ciphertexts in AES-GCM (Section 5.1), as well as optimisations that make it more amenable to 2PC evaluation of the encryption and decryption procedures (Section 5.2). We briefly recall how AES-GCM operates.

AES-GCM Encryption. Let k and IV refer to an encryption key and initialisation vector. Given as input a sequence of n appropriately padded plaintext blocks $M = (M_1, \dots, M_n)$, AES-GCM applies counter-mode encryption to produce the ciphertext blocks $C_i = M_i \oplus \text{AES.Enc}(k, IV + i)$. To ensure authenticity, the algorithm outputs a tag $\tau = \text{Tag}_k(A, C, k, IV)$ computed over C and any associated data A (i.e. any data that is authenticated but not encrypted [63], e.g. protocol headers or metadata) as follows:

- Given some vector $\mathbf{x} \in \mathbb{F}_{2^{128}}^m$, we define the polynomial $P_{\mathbf{x}} = \sum_{i=1}^m x_i \cdot h^{m-i+1}$ over $\mathbb{F}_{2^{128}}$.
- Assuming that C and A are properly padded, we compute τ (where $h = \text{AES.Enc}(k, 0)$) as: $\tau(A, C, k, IV) = \text{AES.Enc}(k, IV) \oplus P_{A||C||\text{len}(A)||\text{len}(C)}(h)$.

5.1. Commitment to AES-GCM Ciphertexts

In DiStefano, both $\mathcal{C}$ and $\mathcal{V}$ learn all AES-GCM ciphertext blocks $C_i = M_i \oplus \text{AES.Enc}(k, IV + i)$ produced by $\mathcal{S}$, where the session key $k = k_c + k_v$ is secret-shared across both $\mathcal{C}$ and $\mathcal{V}$. We briefly describe how $\mathcal{C}$ can commit to the received ciphertext blocks C_i without revealing their key share k_c . Note first that the use of AES-GCM in an AEAD setting leads to a non-committing cipher [64], meaning that an adversary in possession of a key k and a valid ciphertext block $C_i = \text{AES.Enc}(k, IV + i) \oplus M_i$ can find a distinct $k' \neq k$ such that $C_i = M_i' \oplus \text{AES.Enc}(k', IV + i)$ is a valid ciphertext. From the perspective of DCTLS protocols, this non-committing nature of the algorithm presents a challenge, as $\mathcal{C}$ typically only proves statements after learning the entirety of the key. We circumvent the issue as follows.

Assume that $\mathcal{C}$ receives a single tuple of an AES-GCM ciphertext and tag, (C_i, τ) , from $\mathcal{S}$ that they wish to decrypt. As $\mathcal{C}$ only holds a share (k_c) of k , $\mathcal{C}$ cannot decrypt C_i by themselves. Thus, $\mathcal{C}$ forwards (C_i, τ) to $\mathcal{V}$, and they engage in a maliciously-secure 2PC protocol to validate τ on C_i (Algorithm 2). If it succeeds, then both $\mathcal{C}$ and $\mathcal{V}$ are convinced that C_i is a valid ciphertext under k . Yet, we must be careful how we reveal $M_i = \text{AES.Enc}(k, IV + i) \oplus C_i$ to each party as revealing M_i to $\mathcal{C}$ allows them to mount the outlined non-committing attack, while revealing M_i to $\mathcal{V}$ would violate the privacy guarantees of DCTLS. In order to resolve this issue, we use a modified 2PC AES decryption protocol that, after checking that the client input masks match the commitments held by $\mathcal{V}$ and validating τ , outputs $e_i = \text{AES.Enc}(k, IV + i)$ to $\mathcal{C}$ and a commitment, E_i , to e_i to $\mathcal{V}$. With this, notice that $\mathcal{C}$ is unable to exploit the non-committing nature of AES-GCM as a binding commitment to e_i is created and validated by $\mathcal{V}$. Moreover, $\mathcal{C}$ can now reveal individual blocks to $\mathcal{V}$ without requiring either party to reveal their key share: $\mathcal{C}$ can reveal a particular block C_i by simply forwarding e_i to $\mathcal{V}$. In other words, this tweak allows $\mathcal{C}$ to engage in a *selective opening* protocol with $\mathcal{V}$ (we provide details of this technique in Appendix G). Producing these set of commitments is cheap as, in practice, we simply require $\mathcal{C}$ to commit to n unique masks b_i and

then output $E_i = \text{AES.Enc}(k, IV + i) \oplus b_i$. We formalise this scheme in Section D.2. Checking random-oracle commitments in 2PC is practical. Using a low-depth hash function, such as LowMC [65], this would cost 4370 AND gates for the full commitment check [66], which is cheaper than a single AES block evaluation (6400 AND gates). Above all, the practical costs of the 2PC commit scheme are low, and we demonstrate this using a higher-depth, AES-based hash (see Section 7).

5.2. 2PC Performance Optimisations

We now discuss some cryptographic optimisations that are necessary for ensuring the high performance of AES-GCM encryption/decryption during online TLS operations. The ideal functionalities that we use to describe AES-GCM in 2PC and the security proofs of the optimisations are given in Section D.1.

Efficiency. Despite its simplicity, executing AES-GCM encryptions in a multi-party setting can be challenging due to the use of binary and arithmetic operations. For example, whilst AES operations are well-suited for garbled circuits, a single multiplication over $\mathbb{F}_{2^{128}}$ typically requires around 16K AND gates, increasing the cost by nearly a factor of 3. To mitigate this cost, both [2] and [12] recommend computing shares of the powers of h (denoted as $\{h^i\}$) during an offline setup stage, amortising the cost across the entire session. In certain settings, this cost can be reduced further by restricting how many powers of h are used: for example, MPCAuth [18] employs a clever message slicing strategy to minimise the value of i with the assumption that servers only send rather small ciphertexts (e.g., at most 4KiB). However, TLS 1.3 servers can send ciphertexts bigger than that in order to avoid overhead: web servers, for example, that deliver video stream [67] or large files [68] might send responses significantly larger than 4KiB, requiring more ciphertext blocks to be handled. It is recommended [69], for example, that when the connection congestion window is large and a large stream (e.g., streaming video) is transferred, that the size of the TLS 1.3 record should be increased to span multiple TCP packets (up to 16KiB) to reduce framing and CPU overhead. There is no “optimal” record size, but rather it is dynamically adjusted based on the state of the TCP connection. Due to this, in DiStefano we allow for this flexibility, as we explicitly target the largest possible TLS ciphertext of 16KiB, which corresponds to $i = 1024$.

Assuming that a sharing $(\{h_c^i\}, \{h_v^i\})$ exists, producing tags in 2PC is straightforward: tagging n blocks requires two local polynomial evaluations (writing $\tau_c = P_{A||c||\text{len}(A)||\text{len}(c)}(\{h_c^i\})$ and $\tau_v = P_{A||c||\text{len}(A)||\text{len}(c)}(\{h_v^i\})$, respectively) over $\mathbb{F}_{2^{128}}$ and $n + 1$ 2PC evaluations of AES [2], [12]. The final tag is achieved by simply computing $\tau = \tau_c + \tau_v \oplus \text{AES.Enc}(k_c + k_v, IV_c)$. In order to make this more efficient, it is necessary to initially construct a 2PC protocol that evaluates the ciphertext c and outputs to both parties, and then have a subsequent protocol that computes the tag for this ciphertext, based on the local polynomials submitted by the client.

Our optimisations. DECO gives few details on how to compute shares of the powers of h , other than that they are computed in a 2PC session. We remark that calculating these shares in a garbled circuit is unlikely to be feasible: our adapted version of MPCAuth’s share derivation circuit contained around 17M AND gates, and required over 900MiB and 18GiB of network traffic and memory, respectively, just for the pre-processing stage. For comparison, our circuits for TLS 1.3 secret derivation contain around 1.3M AND gates in total, which is approximately a factor of 14 smaller. Thus, using only garbled circuits is unlikely to be feasible.

Several other approaches exist for computing the shares of $\{h^i\}$. For instance, PageSigner [12] reduces computing additive shares of h^i to simply computing shares using MtA computations. Given an initial additive sharing $h = h_c + h_v$, $\mathcal{C}$ and $\mathcal{V}$ iteratively compute additive shares of $\ell_c + \ell_v = h^n = (h_c + h_v)^{n-1}(h_c + h_v)$ for $1 < n \leq 1024$. This approach permits an additional optimisation: as $(x + y)^2 = x^2 + y^2$ over $\mathbb{F}_{2^{128}}$, each party can compute shares of even powers of h locally. Taking this optimisation into account, producing shares in this way costs a total of 1022 MtA operations. However, as computing shares of any odd h^i requires first computing shares of h^{i-2} , the approach seems to require around 500 rounds, which is likely too slow for a WAN setting.

We improve upon this by replacing the additive sharing $h = h_1 + h_2$ with $h = h_1/h_2$, i.e. using a multiplicative sharing. By using multiplicative shares, we can run each MtA computation in parallel, with each P_i supplying $h_i, h_i^3, \dots, h_i^{1023}$ as input. This optimisation asymptotically halves the number of MtA operations and reduces the round complexity to a single round, as the only costly operation is the share’s computation, which is not carried out inside the garbled circuit. However, this tweak does require a slightly more complicated scheme for computing the initial sharing of h , as we now must compute multiplication over $\mathbb{F}_{2^{128}}$, taking the size of the circuit for deriving the initial shares to around 23K AND gates. In practice, we reduce the size of this circuit to around 18K AND gates by instead using a carry-less Karatsuba [70] algorithm. Whilst this still represents an increase of around a factor of 3 compared to the additive circuit, the reduction in MtA operations and rounds means that we are able to achieve an end-to-end speed-up of around a factor of 3. We discuss these results in more detail in Section 7.

6. Security Analysis

Previous DCTLS protocols use all-encompassing ideal functionalities and Universally-Composable (UC) security proofs [71], proving that the entire flow from handshake to attestation is secure. This is problematic for cryptographic agility, as it means that any modification to the TLS ciphersuite, 2PC functionality, or protocol extensions would necessitate a complete rewrite of the proof. Such agility is critical for building flexible secure systems, that can be modified easily even if primitives and systems change [35].

We reimagine the security model for DCTLS protocols in two ways. First, our analysis breaks the protocol down into three phases: handshake, query, and commitment; and guarantees security of each independently. Second, where possible, we adapt the security analysis to either the *standalone* or *game-based* security models. UC security proofs are particularly useful when building atomic protocol primitives, intended for arbitrary composition with other primitives. Since DCTLS is a high-level protocol that is likely to be used as a single application, we believe that our approach captures a natural security requirement, without the added complexity for ensuring UC security. Without this complexity, modifying both individual phases of the protocol and cryptographic primitives is an easier task for future work. We give a short overview of how we model security for each phase in the following. The full standalone security model is covered in Appendix E.

Handshake phase. We model the handshake phase as in the work of Oblivious TLS [17]. Essentially, this model proves that we can satisfy the original guarantees proven about TLS 1.3 [29] (i.e. related to *Match* and *Multi-Stage* security) even when executing certain functionalities in 2PC. One key difference noted by [17] is that the presence of the verifier means that potential TLS 1.3 adversaries can alter the derivation of secrets in the client, and thus security is based on a modified *Shifted PRF ODH* assumption, see [17, Definition 2] for more details. Oblivious TLS produces a UC-security proof for the handshake phase of the protocol, based on maliciously-secure 2PC primitives, and two parties. In our setting, we have an information imbalance, in that $\mathcal{V}$ has fewer powers than one of the two parties analysed in [17], as *only* $\mathcal{C}$ interacts with $\mathcal{S}$ directly, while $\mathcal{V}$ does not interact with $\mathcal{S}$ directly at all. As a result, the analysis of [17] is sufficient for covering our required security guarantees, as long as each 2PC primitive is maliciously-secure. We provide an overview of the protocol execution in Fig. 7, and a more detailed analysis of security in Section E.1.

Query execution phase. The query phase of DiStefano essentially amounts to considering a 2PC realisation of the record-layer of the TLS 1.3 protocol. We define 2PC ideal functionalities (Algorithms 6 and 7) that abstract the core encryption and decryption functionality for application traffic. We show that we can prove security of this phase based on the 2PC realisation of the AES-GCM functionalities that we formalise in Section D.1 (Algorithms 1 and 2), that together implement the functionality and optimisations described in Section 5. To prove security of alternative ciphersuites, it is simply a matter of implementing the 2PC ideal functionalities using different primitives.

Commitment phase. We model the commitment phase in a game-based security model (Section E.3). We provide multiple security notions (Figure 8) that evaluate the capacity of the protocol to satisfy: *session privacy* (SPriv), i.e. committed sessions are indistinguishable; *1-out-of- N authentication* (SAuth _{n} ¹), i.e. the client is forced to successfully authenticate the TLS server amongst N possible apriori-chosen servers; and *session unforgeability* (SUnf), i.e. the client cannot

arbitrarily forge sessions that did not occur. In the end, we show that DiStefano satisfies these properties based on the commitment scheme devised from AES-GCM (Section D.2), and the zero-knowledge proof scheme (ZKPVS) for valid TLS signatures [61] (Section B.3).

7. Experimental Analysis

Implementation. In order to enable easy integration with other cryptographic libraries and browsers, we implemented a full prototype of DiStefano in C⁺.⁸ This implementation contains around 14k lines of code, tests and documentation, and implements the whole protocol. We developed this implementation using C⁺ best practices, and we hope that this effort is useful for other researchers. Concretely, our implementation of DiStefano uses BoringSSL for TLS functionality and emp for all MPC functionality. BoringSSL is the only cryptographic library supported by Chromium-based Internet browsers. As far as we are aware, our implementation contains primitives and circuits that are not available elsewhere. Our implementation also contains a modified version of MPCAuth’s circuit generation to produce the relevant garbled circuits. We further reduce the online cost of MPCAuth’s secret sharing scheme by using a pre-determined splitting scheme for specific secrets.⁹ We list the changes made alongside our prototype¹⁰, and discuss them in Appendix J. Note that further performance improvements, including multithreading, are not addressed in this implementation and are the subject of future work.

Results. We evaluated the performance of DiStefano in LAN and WAN settings. For the LAN environment, we use a consumer-grade device (a Macbook air M1 with 8 GB of RAM) for $\mathcal{C}$, and a server-grade device (an Intel Xeon Gold 6138 with 32 GB of RAM) for $\mathcal{V}$ and $\mathcal{S}$. All communication used in TLS 1.3 was carried out using a single thread over a 1 Gbps network with a latency of around 16 ms. For the WAN setting, we provide various settings depending on regions. We place $\mathcal{C}$ in Paris, France using a AWS EC2 “t.2.xlarge” machine for all settings, but locate the AWS EC2 “t.2.xlarge” machine for $\mathcal{V}$ and $\mathcal{S}$ in three regions: Ohio, USA; London, UK; and Seoul, South Korea. We report their latency based on [72]. For the first region, the median round-trip latency is estimated at 94.15 ms as reported by 13 measurements, and the first and third quartiles are 92.83 ms and 95.53 ms, respectively. For the second region, the median round-trip latency is estimated at 9.07 ms as reported by 13 measurements, and the first and third quartiles are 2.08 ms and 3.31 ms, respectively. For the third region, the median round-trip latency is estimated at 247.91 ms as reported by 13 measurements, and the first and third quartiles are 246.57 ms and 248.87 ms, respectively. Timings and bandwidth measurements are computed as the

8. <https://github.com/brave-experiments/DiStefano>

9. This approach is less flexible than MPCAuth. For example, DiStefano only supports 2 parties, whereas MPCAuth supports arbitrarily many.

10. <https://github.com/brave-experiments/DiStefano>, section “Changes introduced”.

Table 1. GARBLED CIRCUIT TIMINGS AND BANDWIDTH.

Circuit	OT	Offline	Online	Bandwidth
AES-GCM share (K)	LD	2340	34.92	21.04
AES-GCM share (K)	FC	2683	59.48	9.009
AES-GCM share (N)	LD	2678	39.09	25.63
AES-GCM share (N)	FC	2853	61.36	10.35
AES-GCM Tag	LD	1019	22.30	7.604
AES-GCM Tag	FC	2336	22.22	5.010
AES-GCM Verify	LD	1032	21.16	7.746
AES-GCM Verify	FC	2277	21.24	5.130
TLS 1.3 HS (<i>P256</i>)	LD	51470	93.16	305.1
TLS 1.3 HS (<i>P256</i>)	FC	19847	88.90	113.3
TLS 1.3 HS (<i>P384</i>)	LD	51610	95.38	305.8
TLS 1.3 HS (<i>P384</i>)	FC	19940	89.88	113.6
TLS 1.3 TS	LD	51450	95.21	243.7
TLS 1.3 TS	FC	18820	99.25	91.12
2PC-GCM (<i>256B</i>)	LD	18690	82	131.4
2PC-GCM (<i>256B</i>)	FC	10971	80	47.5
2PC-GCM (<i>512B</i>)	LD	31534	136	206.5
2PC-GCM (<i>512B</i>)	FC	16010	142	77.75
2PC-GCM (<i>1KiB</i>)	LD	57485	252	409.4
2PC-GCM (<i>1KiB</i>)	FC	26154	301	151.2
2PC-GCM (<i>2KiB</i>)	LD	114820	728	815.4
2PC-GCM (<i>2KiB</i>)	FC	48764	763	299.3

Each garbled circuit is reported in terms of offline/online times (ms) and total bandwidth (MiB) costs. “K” means Karatsuba and “N” means Naive. “LD” refers to “LeakyDeltaOT” and “FC” means “FerretCOT”. Optimal cases are highlighted in green.

mean of 50 samples, and are represented in milliseconds and mebibytes, respectively (1 MiB is 2^{20} bytes).

Table 1 gives results for each individual circuit used in DiStefano. Each circuit is evaluated without amortization, meaning these timings do not benefit from the amortized pre-processing available in emp. We note that the 2PC-GCM circuit includes both encryption and decryption of traffic, as specified in Algorithms 1 and 2, using a random-oracle (AES-based) commitment scheme. As the most expensive operation of these circuits will only be used once per session, we do not expect that employing amortisation will yield a substantial speed-up. However, employing amortisations for common operations (e.g. AES-GCM tagging and verification) may lead to faster running times (see [48, §7] for concrete speed-ups). We also compare the offline time using the original implementation of authenticated garbling (LeakyDeltaOT [48]) against FerretCOT. Our results show that FerretCOT outperforms the original OT for large circuit sizes in both bandwidth and running time. However, for smaller circuits it appears that the original OT is faster at the cost of more bandwidth. Given that the pre-processing times are proportional to the size of the circuits, our results appear to be predominantly network bound. The results also highlight that our Karatsuba-based circuit achieves modest gains in both bandwidth and time over the naive circuit.

Table 2 shows the results for each arithmetic primitive used. The running times and bandwidth usage are notably low, suggesting that these primitives will not pose a bottleneck, even in constrained network environments. Notably, the tweak introduced in Section 5.2 reduces the running time by a factor of around 3, whilst also halving the required

Table 2. PRIMITIVE TIMINGS AND BANDWIDTH.

Primitive (algorithm)	Time (ms)	Bandwidth (MiB)
ECtF (<i>P256</i>)	336.1	0.768
ECtF (<i>P384</i>)	335.5	1.295
ECtF (<i>P521</i>)	421.4	2.442
MtA (<i>P256</i>)	33.67	0.086
MtA (<i>P384</i>)	40.65	0.127
MtA (<i>P521</i>)	55.83	0.241
AES-GCM powers (mul.)	1694	0.049
AES-GCM powers (add.)	5926	0.080
AES-GCM powers (GC)	—	900

Table 3. E2E TIMINGS (MS) AND BANDWIDTH (MiB) FOR DCTLS IN LAN SETTINGS (WITH LATENCY ≈ 16 ms).

Process	LAN (ms)	Bandwidth (MiB)
<i>Offline costs</i>		
<i>C/S</i> Key Share	1.3167	6.67572e-05
<i>C/V</i> execute ECtF	0.008083	9.53674e-07
Circuit Preprocessing	6280.08	220.484
<i>Online costs</i>		
<i>S</i> sends cert.	0.011375	3.14713e-05
Derive traffic secrets	33.1389	0.0276108
Derive GCM shares	136.573	0.0488291

bandwidth for the multiplication (this ignores bandwidth used by shared setup). This all represents an improvement of around 4 orders of magnitude over using a garbled circuit.

The timings indicate that our implementation of DiStefano is competitive with DECO, with the DCTLS online portion taking ≈ 500 ms to complete for a 256-bit secret in a LAN setting. These conclusions are consistent across both the individual and E2E timings (cf. Table 3). Our WAN experiments model (as seen in Table 4) realistic E2E executions in different regions of the world with low to high latency. The results indicate that the running times roughly increase by the amount of latency introduced. The only deviations occurred are related to either circuit preprocessing — which increases it, but is an offline amortisable cost — and the derivation of GCM shares — which reflects the multi-round trip time nature of this part of the protocol. Even so, the GCM shares derivation still takes less than a second, and the total online costs are significantly lower than standard online TLS handshake timeout times which, while configurable, are typically between 10 and 20 seconds [24]. Finally, we note that increasing latencies only appear to impact the protocol sublinearly, and thus we expect that DiStefano across a variety of browsing scenarios.

7.1. Comparisons with prior work

DECO-like protocols. We note that comparisons between our results and previous DECO-like protocols for committing to TLS 1.2 traffic [2], [12] should be made carefully. In the case of DECO, their implementation is not publicly available, and we were unable to reproduce any of their results. Moreover, as our implementation is single-threaded,

Table 4. E2E TIMINGS (MS) AND BANDWIDTH (MiB) FOR DCTLS IN WAN SETTINGS. ALL TIMES ARE REPORTED IN MS.

Process	WAN-Ohio	WAN-London	WAN-Seoul
<i>Offline costs</i>			
C/S Key Share	102.4682	12.2567	252.1662
C/V execute ECtF	112.6829	2.5680	285.9367
Circuit Preprocessing	8958.3445	6236.5134	10417.6417
<i>Online costs</i>			
S sends cert.	125.9581	2.3673	12.2567
Derive traffic secrets	130.9039	43.6089	273.2959
Derive GCM shares	494.3445	149.2039	872.4691

we are unable to take advantage of **emp**’s multi-threaded pre-processing. Given that [48, §7] reports an order of magnitude increase in bandwidth due to multi-threading, it is not surprising that our offline times are an order of magnitude higher. However, our online timings are comparable with DECO, and parallelising the pre-processing stage would likely mitigate any discrepancies.¹¹ As pre-processing can be carried out before, we do not consider this a major issue. It is also difficult to compare our timings to PageSigner. Their original implementation is written entirely in Javascript, preventing the usage of dedicated hardware resources. Given that our implementation is instead written in C++, we might expect DiStefano to be faster. PageSigner also follows a semi-honest security model and targets TLS 1.2, which are incompatible with DiStefano.

Janus. The work of Janus [15] builds similar DCTLS functionality for certain ciphersuites of TLS 1.3. Their reported timings assume a latency of 0 ms, which fails to model the impact of network bandwidth on protocol runtimes. As such, we can only make coarse-grained analysis of their handshake and record-layer functionality, for their reported 256 B queries 2 kB responses. The overall online communication is 1072 KiB, while offline communication is 320 MiB. In DiStefano, the online communication is 76 KiB, while offline communication is 220.5 MiB. In terms of runtime, Janus reports around 2 s of offline costs, and 1.55 s of online costs (with latency of 0 ms) when run on an Apple Macbook M1 processor. For DiStefano, in the LAN setting (with latency estimated at 16 ms), offline runtimes are 6.28 s, which is clearly slower than the claims of Janus. However, the online runtimes are significantly quicker: around 0.1 s. This provides much more flexibility in the critical phase of the protocol, which is prone to server timeouts, while the offline phase can be performed at any moment. We reinforce that this comparison is not completely accurate as Janus does not report a full LAN/WAN simulation, where the impact of network latency is included. Finally, as explained in Appendix K, Janus shifts the verification of the 2PC functionality to later stages of the protocol, which can lead to violations of the malicious security model. DiStefano, in comparison, is proven secure in a fully malicious setting.

11. Notably, [2] does not mention if the pre-processing is multi-threaded.

8. Discussion

8.1. Related Work

As noted in Section 2, DiStefano is an instance of a DCTLS protocol. Other alternatives exist, but all have limitations as noted in Section 2.2. We summarise the comparison in Table 5, and discuss further below.

The DECO and PageSigner protocols, for example, only (formally) work for TLS 1.2 and under, and provide limited privacy. TownCrier [13] has similar problems, and requires using trusted computing functionality. Recently, the PECO protocol [38] was proposed, which informally extends the DECO protocol to support TLS 1.3, but provides no formal guarantees nor implementation of it.

MPCAuth [18] allows a user to authenticate to N servers independently by doing the work of only authenticating to one. An N -for-1 authentication system consists of many servers and users. Each user has a number of authentication factors they can use to authenticate. The user holds a secret s that they wish to distribute among the N servers. The protocol consists of two phases. In the *enrollment* phase, the user provides the servers with a number of authentication factors, which the servers verify using authentication protocols: these protocols use a mechanism called “TLS-in-SMPC” that allows N servers to jointly act as a TLS client endpoint to communicate with another TLS server. A single server from the N authorised cannot decrypt any TLS traffic, and, after authenticating with these factors, the client secret-shares s and distributes the shares across the servers. In the *authentication* phase, the user runs the MPCAuth protocols for the authentication factors and, once it is authenticated, the N servers can perform computation over s for the user, which is application-specific (such as key recovery, for instance).

The *Oblivious TLS* protocol [17] allows for any TLS endpoint to obliviously interact with another TLS endpoint, without the knowledge that it is interacting with a multi-party computation instance. It consists of the following phases: i) *Multi-Party Key Exchange*, which is the key exchange phase of the TLS handshake ran in an MPC manner by performing an exponentiation between a known public key and a secret exponent, where the output remains secret; ii) *Threshold Signing*, which is the authentication phase of the TLS handshake done by having the TLS transcript signed with EdDSA Schnorr-based signatures in a threshold protocol; and iii) *Record Layer* which is ran by using authenticated encryption, based on AES-GCM, inside MPC.

Recent work on zero-knowledge middleboxes for TLS 1.3 traffic [16] has many similarities with techniques used in DCTLS protocols. However, the verifier is considered to be an on-path proxy that receives and forwards encrypted traffic between the parties (similar to the proxy model of DECO [2]). Furthermore, the client only produces commitments to their own traffic, rather than the traffic received from the server. Applications include increased corporate oversight and enactment of Internet browsing policies

Table 5. COMPARISON OF DCTLS-LIKE PROTOCOLS. [†]SEE SECTION 8.1

Protocol	TLS 1.3	Attest	Ring auth
DECO-like [2], [13]	✗	✓	✗
MPCAuth [18]	✓	✗	✗
Oblivious TLS [17]	✓	✗	✗
ZKMiddleboxes [16]	✓	$\mathcal{C} \rightarrow \mathcal{S}$	✗
Janus [15]	✓ [†]	✓	✗
DiStefano	✓	✓	✓

to be enforced by middleboxes, which are naturally thwarted when all client traffic is sent encrypted over TLS.

Finally, the Janus protocol [15] targets TLS 1.3, but with the limitation that the client must fix a ciphersuite apriori, while DiStefano allows for full negotiation of any TLS-supported ciphersuite. In addition, the Janus security model leaves open the possibility of theoretical attacks by malicious actors, see Appendix K for more details.

Concurrent work. Xie et al. [14] propose a series of optimisations to the MPC protocols used inside DECO, targeting TLS 1.2. Whilst most of these improvements are orthogonal to our work, one interesting optimisation is a faster approach for deriving TLS traffic secrets inside garbled circuits. This approach is reminiscent of the highly optimised CBC-HMAC protocol proposed in DECO [2, §4.2.1] for computing tags in 2PC. We remark that incorporating this particular optimisation into our secret derivation process seems non-trivial: we discuss these difficulties in Appendix H.

8.2. Applications

Attestations. DiStefano produces commitments to encrypted TLS 1.3 data which, as noted in [2], can be used as the basis of zero-knowledge proofs (or *attestations*) for showing that certain facts are present in such data. However, such attestations can also be constructed via different methods, using cooperative decryption of certain ciphertext blocks, or more generic 2PC techniques. The DECO protocol provides examples that they can prove certain statements for, including proof of confidential financial information, and proof of age. It should be noted that TLS sessions could serve as the basis for more generic user credentials, proving arbitrary facts about a user. For a complete summary, see [2].

Concrete applications of DiStefano include: i. leveraging attestations to produce strong signals of anti-fraudulent behavior by attesting the inclusion of statements over bank account balances or proofs of transactions, which helps prove that a user is not engaging in fraud, as such signals are expensive for bots to replicate; ii. using attestations to verify “real” events by producing statements that confirm the origin and trustworthiness of data, helping to demonstrate that it was not generated by Artificial Intelligence (AI); and iii. attesting to a user’s honesty (by exporting multiple TLS attestations) over a long time period, to grant them access to other privacy-preserving protocols [73], [74].

Table 6. RESULTS FOR RUNNING KeyUpdate IN 2PC.

Circuit	OT	Offline (ms)	Online (ms)	Bandwidth (MiB)
KeyUpdate	LD	10540	29.95	98.54
KeyUpdate	FC	7960	31.96	31.61

8.3. Limitations

Our implementation of DiStefano does not support key rotation via KeyUpdate messages or full 0-RTT mode, but this limitation is not major: it can be circumvented by simply re-running the HSP.¹² We also provide no concrete instantiation of the zero-knowledge primitives that can be used to create attestations, but they should follow the guidelines stated in Section 4.4. Said proofs must also be mindful of user privacy concerns: if proof circuits explicitly target server-specific HTML formats, this will undo the zero-knowledge authentication privacy guarantees of the ZKPVS approach. Note also that the ZKPVS scheme preserves anonymity only amongst the set R : we address this point in Section B.3.

DCTLS protocols assume user commitments are *meaningful*, and that $\mathcal{S}$ stores only *correct* data. Suppose a user wants to provide a proof of their age from a government agency website. They will log in to the website and then run DiStefano to produce a commitment to their age, based on the data present. This process assumes the authentication process is sound, which may not be the case (if the account is stolen or fake). $\mathcal{V}$ should only accept commitments from a $\mathcal{S}$ that it trusts to correctly store user data.

It is important to note that DCTLS protocols could become actively harmful tools for monitoring or censoring client traffic in certain applications, especially in automated systems without human involvement. For instance, they could be exploited to scan and censor specific statements posted by digital activists on social media in order to censor them, which is a known technique used around the world [75], [76]. Thus, we would like to emphasise that deployment of tools such as DiStefano must be considered carefully in such contexts. Furthermore, DCTLS can be subject to different legal and compliance issues if considered as a form of webscraping. The compliance of DCTLS tools with a given website’s terms of service, for example, is an application-specific question in their legal context.

8.4. Browser Integration

DiStefano can be integrated into any browser that uses BoringSSL, e.g. Google Chrome/Brave, easily. As our changes to BoringSSL itself are rather minimal, it would be possible to describe our changes as a series of deltas in a version control system, which can then be applied during the process of building the browser based on build flags.¹³ We leave the completion and deployment as future work.

¹² For completeness, we benchmarked the cost of running the KeyUpdate operation in a garbled circuit, see Table 6.

¹³ Indeed, such a system is already used for the Brave Browser.

9. Conclusion

We build DiStefano, a DCTLS protocol that generates private commitments to encrypted TLS 1.3 data. We use a modular, standalone security framework that provides malicious security, and guarantees privacy for client browsing patterns amongst pre-approved servers. We provide an open-source integration in BoringSSL, and demonstrate the online efficiency of DiStefano for believable workloads.¹⁴ The flexibility, security, and deployability of DiStefano makes it an immediate candidate for real-world applications.

Acknowledgement

We would like to thank Xiao Wang for his help with the various aspects of EMP and for sharing an initial integration of Ferret; Jonathan Hoyland, Yashvanth Kondi and Peter Scholl for useful conversations; Benjamin Livshits for discussing interesting applications for this work. We thank Ryan Little, Daniel S. Roche, and Mayank Varia for pointing out the replay attack on the protocol and for the discussions on the mitigation. Large parts of this work were completed while all of the authors were affiliated with Brave Software. Joe Rowell was supported by EPSRC grant EP/P009301/1.

References

- [1] E. Rescorla, “The transport layer security protocol version 1.3,” Internet Requests for Comments, RFC Editor, RFC 8446, August 2018.
- [2] F. Zhang, D. Maram, H. Malvai, S. Goldfeder, and A. Juels, “DECO: Liberating web data using decentralized oracles for TLS,” in *ACM CCS 2020*, J. Ligatti, X. Ou, J. Katz, and G. Vigna, Eds. ACM Press, Nov. 2020, pp. 1919–1938.
- [3] M. Rosenberg, J. White, C. Garman, and I. Miers, “zk-creds: Flexible anonymous credentials from zkSNARKs and existing identity infrastructure,” Cryptology ePrint Archive, Report 2022/878, 2022. [Online]. Available: <https://eprint.iacr.org/2022/878>
- [4] J. T. Cross, “Age verification in the 21st century: Swiping away your privacy,” *J. Marshall J. Computer & Info. L.*, vol. 23, p. 363, 2004.
- [5] B. Szoka and A. D. Thierier, “Coppa 2.0: The new battle over privacy, age verification, online safety & free speech,” *Progress & Freedom Foundation Progress on Point Paper No.*, vol. 16, 2009.
- [6] M. Yar, “Protecting children from internet pornography? a critical assessment of statutory age verification and its enforcement in the uk,” *Policing: An International Journal*, vol. 43, no. 1, pp. 183–197, 2020.
- [7] P. Blake, “Age verification for online porn: more harm than good?” *Porn Studies*, vol. 6, no. 2, pp. 228–237, 2019.
- [8] R. S. Lear and J. D. Reynolds, “Your social security number or your life: Disclosure of personal identification information by military personnel and the compromise of privacy and national security,” *BU Int’l LJ*, vol. 21, p. 1, 2003.
- [9] J. J. Darrow and S. D. Lichtenstein, “Do you really need my social security number-data collection practices in the digital age,” *NCJL & Tech.*, vol. 10, p. 1, 2008.
- [10] European Commission, “GDPR: Right to Portability, Art. 20,” <https://gdpr-info.eu/art-20-gdpr/>. Accessed 5th September 2023., 2014.
- [11] A. Guy, M. Sporny, D. Reed, and M. Sabadello, “Decentralized identifiers (DIDs) v1.0,” W3C, W3C Recommendation, Jul. 2022, <https://www.w3.org/TR/2022/REC-did-core-20220719/>.
- [12] P. Team, “PageSigner: One-click website auditing,” Website, 2023, <https://old.tlsnotary.org/pagesigner>. Accessed 04/04/2023.
- [13] F. Zhang, E. Cecchetti, K. Croman, A. Juels, and E. Shi, “Town crier: An authenticated data feed for smart contracts,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’16. New York, NY, USA: Association for Computing Machinery, 2016, p. 270–282. [Online]. Available: <https://doi.org/10.1145/2976749.2978326>
- [14] X. Xie, K. Yang, X. Wang, and Y. Yu, “Lightweight authentication of web data via garble-then-prove,” Cryptology ePrint Archive, Report 2023/964, 2023. [Online]. Available: <https://eprint.iacr.org/2023/964>
- [15] J. Lauinger, J. Ernstberger, A. Finkenzeller, and S. Steinhorst, “Janus: Fast privacy-preserving data provenance for tls 1.3,” Cryptology ePrint Archive, Paper 2023/1377, 2023, <https://eprint.iacr.org/2023/1377>. [Online]. Available: <https://eprint.iacr.org/2023/1377>
- [16] P. Grubbs, A. Arun, Y. Zhang, J. Bonneau, and M. Walfish, “Zero-knowledge middleboxes,” in *USENIX Security 2022*, K. R. B. Butler and K. Thomas, Eds. USENIX Association, Aug. 2022, pp. 4255–4272.
- [17] D. Abram, I. Damgård, P. Scholl, and S. Trieflinger, “Oblivious TLS via multi-party computation,” in *CT-RSA 2021*, ser. LNCS, K. G. Paterson, Ed., vol. 12704. Springer, Cham, May 2021, pp. 51–74.
- [18] S. Tan, W. Chen, R. Deng, and R. A. Popa, “MPCAuth: Multi-factor authentication for distributed-trust systems,” in *2023 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2023, pp. 829–847.
- [19] H. Lee, D. Kim, and Y. Kwon, “Tls 1.3 in practice: how tls 1.3 contributes to the internet,” in *Proceedings of the Web Conference 2021*, ser. WWW ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 70–79. [Online]. Available: <https://doi.org/10.1145/3442381.3450057>
- [20] Cloudflare, “TLS 1.2 vs. TLS 1.3 vs. QUIC: Distribution of secure traffic by protocol,” 2023, accessed 11/04/2023. [Online]. Available: <https://radar.cloudflare.com/adoption-and-usage#tls-1-2-vs-tls-1-3-vs-quic>
- [21] D. Tymokhanov and O. Shlomovits, “Alpha-rays: Key extraction attacks on threshold ecdsa implementations,” Cryptology ePrint Archive, Paper 2021/1621, 2021, <https://eprint.iacr.org/2021/1621>. [Online]. Available: <https://eprint.iacr.org/2021/1621>
- [22] N. Makriyannis and U. Peled, “A note on the security of gg18,” 2021, https://info.fireblocks.com/hubfs/A_Note_on_the_Security_of_GG.pdf. [Online]. Available: https://info.fireblocks.com/hubfs/A_Note_on_the_Security_of_GG.pdf
- [23] X. Wang, A. J. Malozemoff, and J. Katz, “Emp-toolkit: Efficient multiparty computation toolkit,” <https://github.com/emp-toolkit>, 2016.
- [24] IBM, “Handshake timer,” 2023, <https://www.ibm.com/docs/en/zos/3.1.0?topic=considerations-handshake-timer>. Accessed 06/02/24.
- [25] D. Adrian, K. Bhargavan, Z. Durumeric, P. Gaudry, M. Green, J. A. Halderman, N. Heninger, D. Springall, E. Thomé, L. Valenta, B. VanderSloot, E. Wustrow, S. Zanella-Béguelin, and P. Zimmermann, “Imperfect Forward Secrecy: How Diffie-Hellman Fails in Practice,” in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’15. New York, NY, USA: Association for Computing Machinery, 2015, pp. 5–17.
- [26] K. Arai and S. Matsuo, “Formal verification of TLS 1.3 full handshake protocol using proverif (Draft-11),” IETF TLS mailing list, 2016. [Online]. Available: <https://mailarchive.ietf.org/arch/msg/tls/NXGYUUXCD2b9WwBRWbvrccjdyI>

14. <https://github.com/brave-experiments/DiStefano>

- [27] N. Aviram, S. Schinzel, J. Somorovsky, N. Heninger, M. Dankel, J. Steube, L. Valenta, D. Adrian, J. A. Halderman, V. Dukhovni, E. Käsper, S. Cohn, S. Engels, C. Paar, and Y. Shavitt, “DROWN: Breaking TLS using SSLv2,” in *25th USENIX Security Symposium (USENIX Security 16)*. Austin, TX: USENIX Association, August 2016, pp. 689–706. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/aviram>
- [28] B. Beurdouche, K. Bhargavan, A. Delignat-Lavaud, C. Fournet, M. Kohlweiss, A. Pironti, P.-Y. Strub, and J. K. Zinzindohoue, “A messy state of the union: Taming the composite state machines of tls,” in *2015 IEEE Symposium on Security and Privacy*, 2015, pp. 535–552.
- [29] B. Dowling, M. Fischlin, F. Günther, and D. Stebila, “A cryptographic analysis of the TLS 1.3 handshake protocol,” *Journal of Cryptology*, vol. 34, no. 4, p. 37, Oct. 2021.
- [30] R. Holz, J. Hiller, J. Amann, A. Razaghpanah, T. Jost, N. Vallina-Rodriguez, and O. Hohlfeld, “Tracking the deployment of tls 1.3 on the web: A story of experimentation and centralization,” *SIGCOMM Comput. Commun. Rev.*, vol. 50, no. 3, p. 3–15, jul 2020. [Online]. Available: <https://doi.org/10.1145/3411740.3411742>
- [31] T. Team, “TLSNotary: Proof of data authenticity,” 2023, <https://tlsnotary.github.io/landing-page/>. Accessed 04/04/2023.
- [32] J. V. Bulck, M. Minkin, O. Weisse, D. Genkin, B. Kasikci, F. Piessens, M. Silberstein, T. F. Wenisch, Y. Yarom, and R. Strackx, “Foreshadow: Extracting the keys to the intel SGX kingdom with transient Out-of-Order execution,” in *27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, Aug. 2018, p. 991–1008. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity18/presentation/bulck>
- [33] H. Ritzdorf, K. Wüst, A. Gervais, G. Felley, and S. Capkun, “TLS-N: Non-repudiation over TLS enable ubiquitous content signing,” in *NDSS 2018*. The Internet Society, Feb. 2018.
- [34] A. Backman, J. Richer, and M. Sporny, “Signing http messages,” IETF draft, accessed 14/11/2022. [Online]. Available: <https://www.ietf.org/archive/id/draft-ietf-httpbis-message-signatures-04.html>
- [35] D. Ott, K. Paterson, and D. Moreau, “Where is the research on cryptographic transition and agility?” *Commun. ACM*, vol. 66, no. 4, p. 29–32, mar 2023. [Online]. Available: <https://doi.org/10.1145/3567825>
- [36] K. Yang, C. Weng, X. Lan, J. Zhang, and X. Wang, “Ferret: Fast extension for correlated OT with small communication,” in *ACM CCS 2020*, J. Ligatti, X. Ou, J. Katz, and G. Vigna, Eds. ACM Press, Nov. 2020, pp. 1607–1626.
- [37] M. Rosulek and L. Roy, “Three halves make a whole? Beating the half-gates lower bound for garbled circuits,” in *CRYPTO 2021, Part I*, ser. LNCS, T. Malkin and C. Peikert, Eds., vol. 12825. Virtual Event: Springer, Cham, Aug. 2021, pp. 94–124.
- [38] M. B. Santos, “Peco: methods to enhance the privacy of deco protocol,” Cryptology ePrint Archive, Paper 2022/1774, 2022, <https://eprint.iacr.org/2022/1774>. [Online]. Available: <https://eprint.iacr.org/2022/1774>
- [39] K. Y. Chan, H. Cui, and T. H. Yuen, “Dido: Data provenance from restricted tls 1.3 websites,” Cryptology ePrint Archive, Paper 2023/1056, 2023, <https://eprint.iacr.org/2023/1056>. [Online]. Available: <https://eprint.iacr.org/2023/1056>
- [40] Y. Lindell and B. Pinkas, “Secure multiparty computation for privacy-preserving data mining,” Cryptology ePrint Archive, Report 2008/197, 2008. [Online]. Available: <https://eprint.iacr.org/2008/197>
- [41] A. C. Yao, “Protocols for secure computations,” in *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, 1982, pp. 160–164.
- [42] O. Goldreich, S. Micali, and A. Wigderson, “Proofs that yield nothing but their validity and a methodology of cryptographic protocol design (extended abstract),” in *27th FOCS*. IEEE Computer Society Press, Oct. 1986, pp. 174–187.
- [43] I. Damgård, V. Pastro, N. P. Smart, and S. Zakarias, “Multi-party computation from somewhat homomorphic encryption,” in *CRYPTO 2012*, ser. LNCS, R. Safavi-Naini and R. Canetti, Eds., vol. 7417. Springer, Berlin, Heidelberg, Aug. 2012, pp. 643–662.
- [44] M. Keller, “MP-SPDZ: A versatile framework for multi-party computation,” in *ACM CCS 2020*, J. Ligatti, X. Ou, J. Katz, and G. Vigna, Eds. ACM Press, Nov. 2020, pp. 1575–1590.
- [45] M. Keller, E. Orsini, and P. Scholl, “MASCOT: Faster malicious arithmetic secure computation with oblivious transfer,” in *ACM CCS 2016*, E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi, Eds. ACM Press, Oct. 2016, pp. 830–842.
- [46] D. Beaver, “Efficient multiparty protocols using circuit randomization,” in *CRYPTO ’91*, ser. LNCS, J. Feigenbaum, Ed., vol. 576. Springer, Berlin, Heidelberg, Aug. 1992, pp. 420–432.
- [47] V. Kolesnikov and T. Schneider, “Improved garbled circuit: Free XOR gates and applications,” in *ICALP 2008, Part II*, ser. LNCS, L. Aceto, I. Damgård, L. A. Goldberg, M. M. Halldórsson, A. Ingólfssdóttir, and I. Walukiewicz, Eds., vol. 5126. Springer, Berlin, Heidelberg, Jul. 2008, pp. 486–498.
- [48] X. Wang, S. Ranellucci, and J. Katz, “Authenticated garbling and efficient maliciously secure two-party computation,” in *ACM CCS 2017*, B. M. Thuraisingham, D. Evans, T. Malkin, and D. Xu, Eds. ACM Press, Oct. / Nov. 2017, pp. 21–37.
- [49] Y. Ishai, J. Kilian, K. Nissim, and E. Petrank, “Extending oblivious transfers efficiently,” in *CRYPTO 2003*, ser. LNCS, D. Boneh, Ed., vol. 2729. Springer, Berlin, Heidelberg, Aug. 2003, pp. 145–161.
- [50] M. Keller, E. Orsini, and P. Scholl, “Actively secure OT extension with optimal overhead,” in *CRYPTO 2015, Part I*, ser. LNCS, R. Gennaro and M. J. B. Robshaw, Eds., vol. 9215. Springer, Berlin, Heidelberg, Aug. 2015, pp. 724–741.
- [51] C. Guo, J. Katz, X. Wang, and Y. Yu, “Efficient and secure multiparty computation from fixed-key block ciphers,” in *2020 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2020, pp. 825–841.
- [52] R. Gennaro and S. Goldfeder, “Fast multiparty threshold ECDSA with fast trustless setup,” in *ACM CCS 2018*, D. Lie, M. Mannan, M. Backes, and X. Wang, Eds. ACM Press, Oct. 2018, pp. 1179–1194.
- [53] P. Paillier, “Public-key cryptosystems based on composite degree residuosity classes,” in *EUROCRYPT ’99*, ser. LNCS, J. Stern, Ed., vol. 1592. Springer, Berlin, Heidelberg, May 1999, pp. 223–238.
- [54] H. Xue, M. H. Au, X. Xie, T. H. Yuen, and H. Cui, “Efficient online-friendly two-party ECDSA signature,” in *ACM CCS 2021*, G. Vigna and E. Shi, Eds. ACM Press, Nov. 2021, pp. 558–573.
- [55] I. Haitner, N. Makriyannis, S. Ranellucci, and E. Tsfadia, “Highly efficient OT-based multiplication protocols,” in *EUROCRYPT 2022, Part I*, ser. LNCS, O. Dunkelman and S. Dziembowski, Eds., vol. 13275. Springer, Cham, May / Jun. 2022, pp. 180–209.
- [56] J. Doerner, Y. Kondi, E. Lee, and a. shelat, “Secure two-party threshold ECDSA from ECDSA assumptions,” in *2018 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2018, pp. 980–997.
- [57] —, “Threshold ECDSA from ECDSA assumptions: The multiparty case,” in *2019 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, May 2019, pp. 1051–1066.
- [58] R. Impagliazzo and M. Naor, “Efficient cryptographic schemes provably as secure as subset sum,” *Journal of Cryptology*, vol. 9, no. 4, pp. 199–216, Sep. 1996.
- [59] R. Little, D. S. Roche, and M. Varia, “Ticket to hide: Private, practical proofs of provenance for TLS,” Cryptology ePrint Archive, Paper 2026/540, 2026. [Online]. Available: <https://eprint.iacr.org/2026/540>
- [60] J. Groth and M. Kohlweiss, “One-out-of-many proofs: Or how to leak a secret and spend a coin,” in *EUROCRYPT 2015, Part II*, ser. LNCS, E. Oswald and M. Fischlin, Eds., vol. 9057. Springer, Berlin, Heidelberg, Apr. 2015, pp. 253–280.

- [61] S. Celi, S. Levin, and J. Rowell, “Cdls: Proving knowledge of committed discrete logarithms with soundness,” *Cryptology ePrint Archive*, Paper 2023/1595, 2023, <https://eprint.iacr.org/2023/1595>. [Online]. Available: <https://eprint.iacr.org/2023/1595>
- [62] A. Faz-Hernández, W. Ladd, and D. Maram, “ZKAttest: Ring and group signatures for existing ECDSA keys,” in *SAC 2021*, ser. LNCS, R. AlTawy and A. Hülsing, Eds., vol. 13203. Springer, Cham, Sep. / Oct. 2022, pp. 68–83.
- [63] D. McGrew, “An interface and algorithms for authenticated encryption,” *Internet Requests for Comments*, RFC Editor, RFC 5116, January 2008.
- [64] P. Grubbs, J. Lu, and T. Ristenpart, “Message franking via committing authenticated encryption,” in *CRYPTO 2017, Part III*, ser. LNCS, J. Katz and H. Shacham, Eds., vol. 10403. Springer, Cham, Aug. 2017, pp. 66–97.
- [65] M. R. Albrecht, C. Rechberger, T. Schneider, T. Tiessen, and M. Zohner, “Ciphers for MPC and FHE,” in *EUROCRYPT 2015, Part I*, ser. LNCS, E. Oswald and M. Fischlin, Eds., vol. 9056. Springer, Berlin, Heidelberg, Apr. 2015, pp. 430–454.
- [66] N. Agrawal, J. Bell, A. Gascón, and M. J. Kusner, “MPC-friendly commitments for publicly verifiable covert security,” in *ACM CCS 2021*, G. Vigna and E. Shi, Eds. ACM Press, Nov. 2021, pp. 2685–2704.
- [67] N. T. Blog, “How netflix brings safer and faster streaming experiences to the living room on crowded networks using tls 1.3,” <https://tinyurl.com/mrydxrsw>, 2020.
- [68] M. Silverlock and G. Redner, “Bringing modern transport security to google cloud with tls 1.3,” <https://cloud.google.com/blog/products/networking/tls-1-3-is-now-on-by-default-for-google-cloud-services>, 2020.
- [69] I. Grigorik, *High Performance Browser Networking: What every web developer should know about networking and browser performance*. O’Reilly Media, Incorporated, 2013. [Online]. Available: https://books.google.pt/books?id=X_SsMQEACAAJ
- [70] S. Gueron and M. E. Konavis, “Intel® carry-less multiplication instruction and its usage for computing the gcm mode,” 2014, accessed 14/03/2023. [Online]. Available: <https://www.intel.com/content/dam/develop/external/us/en/documents/clmul-wp-rev-2-02-2014-04-20.pdf>
- [71] R. Canetti, “Universally composable security: A new paradigm for cryptographic protocols,” in *42nd FOCS*. IEEE Computer Society Press, Oct. 2001, pp. 136–145.
- [72] CloudPing, “Aws latency monitoring,” https://www.cloudping.co/grid/p_25/timeframe/1D, 2024.
- [73] L. Tulloch and I. Goldberg, “Lox: Protecting the social graph in bridge distribution,” *Privacy Enhancing Technologies*, vol. 2023, no. 1, 2023. [Online]. Available: <https://petsymposium.org/popets/2023/popets-2023-0029.pdf>
- [74] L. N. R. Seny Kamara, “Cryptography for grassroots organising,” <https://iacr.org/submit/files/slides/2023/rwc/rwc2023/69/slides.pdf>, 2023.
- [75] G. Awwad and K. Toyama, “Digital repression in palestine,” in *CHI*. ACM, 2024. [Online]. Available: <https://dl.acm.org/doi/pdf/10.1145/3613904.3642422>
- [76] D. Xue, A. Ablove, R. Ramesh, G. K. Danciu, and R. Ensafi, “Bridging barriers: A survey of challenges and priorities in the censorship circumvention landscape,” in *USENIX Security Symposium*. USENIX, 2024. [Online]. Available: <https://www.usenix.org/system/files/usenixsecurity24-xue-bridging.pdf>
- [77] M. Naor and M. Yung, “Public-key cryptosystems provably secure against chosen ciphertext attacks,” in *22nd ACM STOC*. ACM Press, May 1990, pp. 427–437.

Table 7. TLS 1.3 HANDSHAKE AND TRAFFIC SECRETS.

Secret	Context Input	Label
CHTS	$H_0 = \mathcal{H}(\text{CH}, \dots, \text{SH})$	Label ₁ = “c hs traffic”
SHTS	$H_0 = \mathcal{H}(\text{CH}, \dots, \text{SH})$	Label ₂ = “s hs traffic”
dHS	$H_1 = \mathcal{H}(\text{“”})$	Label ₃ = “derived”
fk _S	$H_\epsilon = \mathcal{H}(\emptyset)$	Label ₄ = “finished”
fk _C	$H_\epsilon = \mathcal{H}(\emptyset)$	Label ₄ = “finished”
CATS	$H_2 = \mathcal{H}(\text{CH}, \dots, \text{SF})$	Label ₅ = “c ap traffic”
SATS	$H_2 = \mathcal{H}(\text{CH}, \dots, \text{SF})$	Label ₆ = “s ap traffic”

Table 8. TLS 1.3 AUTH MESSAGES AND ASSOCIATED HASHES, WHERE Label₇ IS “TLS 1.3, SERVER CERTIFICATEVERIFY” AND Label₈ IS “TLS 1.3, CLIENT CERTIFICATEVERIFY”.

Auth message	Context Input	Label
SCV	$H_3 = \mathcal{H}(\text{CH}, \dots, \text{SCRT})$	Label ₇
SF	$H_4 = \mathcal{H}(\text{CH}, \dots, \text{SCV})$	
CCV	$H_5 = \mathcal{H}(\text{CH}, \dots, \text{CCRT})$	Label ₈
CF	$H_6 = \mathcal{H}(\text{CH}, \dots, \text{CCV})$	

- [78] T. Iwata, K. Ohashi, and K. Minematsu, “Breaking and repairing GCM security proofs,” in *CRYPTO 2012*, ser. LNCS, R. Safavi-Naini and R. Canetti, Eds., vol. 7417. Springer, Berlin, Heidelberg, Aug. 2012, pp. 31–49.
- [79] R. L. Rivest, A. Shamir, and Y. Tauman, “How to leak a secret,” in *ASIACRYPT 2001*, ser. LNCS, C. Boyd, Ed., vol. 2248. Springer, Berlin, Heidelberg, Dec. 2001, pp. 552–565.
- [80] K. Q. Nguyen, F. Bao, Y. Mu, and V. Varadharajan, “Zero-knowledge proofs of possession of digital signatures and its applications,” in *ICICS 99*, ser. LNCS, V. Varadharajan and Y. Mu, Eds., vol. 1726. Springer, Berlin, Heidelberg, Nov. 1999, pp. 103–118.
- [81] S. R. Department, “Global number of customers at the largest banks in the united kingdom (uk) in 2022,” Statista, Aug 29 2023, <http://tinyurl.com/statista-banks>.
- [82] M. Bellare and P. Rogaway, “Entity authentication and key distribution,” in *CRYPTO’93*, ser. LNCS, D. R. Stinson, Ed., vol. 773. Springer, Berlin, Heidelberg, Aug. 1994, pp. 232–249.
- [83] F. Günther, “Modeling advanced security aspects of key exchange and secure channel protocols,” *it - Information Technology*, vol. 62, no. 5-6, pp. 287–293, 2020.
- [84] J. Brendel, M. Fischlin, F. Günther, and C. Janson, “PRF-ODH: Relations, instantiations, and impossibility results,” in *CRYPTO 2017, Part III*, ser. LNCS, J. Katz and H. Shacham, Eds., vol. 10403. Springer, Cham, Aug. 2017, pp. 651–681.

Appendix A. TLS 1.3 Handshake

An overview of the TLS 1.3 handshake is given in Fig. 5, our notation is based on the notation defined in [29], which we provide in Table 8 and Table 7 for completeness.

Appendix B. Additional Cryptographic Preliminaries

We provide some additional cryptographic preliminaries that are required for arguing the security of our system.

Figure 5. TLS 1.3 handshake with certificate-based authentication. Short-hands correspond to [29]. Purple represents messages sent or calculated by $\mathcal{C}$, pink by $\mathcal{S}$. Messages with an asterisk (*) are optional, and those within braces ({}) are encrypted.

Client	Server
---------------	---------------

static (Sig): pk_S, sk_S

ClientHello:
 $r_c \leftarrow \{0, 1\}^{256}, x_c \leftarrow \mathbb{Z}_q$
+ClientKeyShare: $X_c \leftarrow g^{x_c}$

ServerHello: $r_s \leftarrow \{0, 1\}^{256}, y_s \leftarrow \mathbb{Z}_q$
+ServerKeyShare: $Y_s \leftarrow g^{y_s}$

$\text{DHE} \leftarrow Y_s^{x_c}$ $\text{DHE} \leftarrow X_c^{y_s}$

$\text{HS} \leftarrow \text{HKDF}.\text{Extract}(\emptyset, \text{DHE})$
 $\text{CHTS} \leftarrow \text{HKDF}.\text{Expand}(\text{HS}, \text{Label}_1 \parallel H_0)$
 $\text{SHTS} \leftarrow \text{HKDF}.\text{Expand}(\text{HS}, \text{Label}_2 \parallel H_0)$
 $\text{dHS} \leftarrow \text{HKDF}.\text{Expand}(\text{HS}, \text{Label}_3 \parallel H_1)$
 $\text{tk}_{chs} \leftarrow \text{DeriveTK}(\text{CHTS})$
 $\text{tk}_{shs} \leftarrow \text{DeriveTK}(\text{SHTS})$

+EncryptedExtensions }
+CertificateRequest } *
+ServerCertificate: } pk_S
+ServerCertificateVerify: }
 $\text{Sig} \leftarrow \text{Sign}(\text{sk}_S, \text{Label}_7 \parallel H_3)$
 $\text{fk}_S \leftarrow \text{HKDF}.\text{Expand}(\text{SHTS}, \text{Label}_4 \parallel H_e)$
+ServerFinished: } $SF \leftarrow \text{HMAC}(\text{fk}_S, H_4)$

$\text{MS} \leftarrow \text{HKDF}.\text{Extract}(\text{dHS}, \emptyset)$
 $\text{CATS} \leftarrow \text{HKDF}.\text{Expand}(\text{MS}, \text{Label}_5 \parallel H_2)$
 $\text{SATS} \leftarrow \text{HKDF}.\text{Expand}(\text{MS}, \text{Label}_6 \parallel H_2)$

+ClientCertificate: } * pk_C
+ClientCertificateVerify: } *
 $\text{Sig} \leftarrow \text{Sign}(\text{sk}_C, \text{Label}_8 \parallel H_5)$
 $\text{fk}_C \leftarrow \text{HKDF}.\text{Expand}(\text{CHTS}, \text{Label}_4 \parallel H_e)$
+ClientFinished: } $CF \leftarrow \text{HMAC}(\text{fk}_C, H_6)$

$\text{tk}_{capp} \leftarrow \text{DeriveTK}(\text{CATS})$
 $\text{tk}_{sapp} \leftarrow \text{DeriveTK}(\text{SATS})$
 $\text{abort if } \text{Verify}(\text{pk}_C, \text{Label}_8 \parallel H_5, \text{Sig}) \neq 1$
 $\text{abort if } CF \neq \text{HMAC}(\text{fk}_C, H_6)$

B.1. Commitment Schemes

Definition 3 (Commitment scheme). A commitment scheme Γ is a tuple consisting of the following algorithms:

- $\Gamma.\text{Gen}(1^\lambda)$: outputs some secret parameters sp ;
- $\Gamma.\text{Commit}(\text{sp}, x)$: outputs a commitment c ;
- $\Gamma.\text{Challenge}(c)$: outputs a random challenge t ;
- $\Gamma.\text{Open}(\text{sp}, c, t, x)$: outputs a bit $b \in \{0, 1\}$.

An interactive commitment scheme, $\tilde{\Gamma}$, between a committer, $\mathcal{C}$, and a revealer, $\mathcal{R}$, proceeds as follows:

- $\mathcal{C}$ runs $\text{sp} \leftarrow \Gamma.\text{Gen}(1^\lambda)$, and sends $c \leftarrow \Gamma.\text{Commit}(\text{sp}, x)$ to $\mathcal{R}$;

- $\mathcal{R}$ sends $t \leftarrow \Gamma.\text{Challenge}(c)$ to $\mathcal{C}$;
- $\mathcal{C}$ sends x to $\mathcal{R}$;
- $\mathcal{R}$ outputs $b \stackrel{?}{=} 1$, for $b \leftarrow \Gamma.\text{Open}(\text{sp}, c, t, x)$.

Definition 4 (Binding property). Given $\text{sp} \leftarrow \Gamma.\text{Gen}(1^\lambda)$. We say that Γ is a computationally binding commitment scheme if, for any PPT algorithm, the following holds:

$$\Pr \left[0 \leftarrow \Gamma.\text{Open}(\text{sp}, c^*, t^*, x') \mid \begin{array}{l} (x^*, c^*) \leftarrow \mathcal{A}(1^\lambda) \\ t^* \leftarrow \Gamma.\text{Challenge}(c^*) \\ x' \leftarrow \mathcal{A}(1^\lambda); x' \neq x^* \end{array} \right] > 1 - \text{negl}(\lambda).$$

We say that Γ is perfectly binding if the same holds for unbounded algorithms, with probability 1.

Definition 5 (Hiding property). Let $\text{sp} \leftarrow \Gamma.\text{Gen}(1^\lambda)$, $\{x_b\}_{b \in \{0, 1\}} \in \{0, 1\}^2$, and $\{c_b \leftarrow \Gamma.\text{Commit}(\text{sp}, x_b)\}$. We say that Γ is a computationally hiding commitment scheme if, for any PPT algorithm, the following holds:

$$\Pr \left[d^* \stackrel{?}{=} d \mid \begin{array}{l} d \leftarrow \{0, 1\} \\ d^* \leftarrow \mathcal{A}(1^\lambda, c_d, (x_0, x_1)) \end{array} \right] < 1/2 + \text{negl}(\lambda).$$

We say that Γ is perfectly hiding if the same holds for unbounded algorithms, with probability 1/2.

We show in Section D.2 that AES-GCM ciphertext commitment scheme in Section 5.1 is perfectly binding and computationally hiding for TLS 1.3 encrypted data. A high-level overview of the commitment phase based on this scheme is given in Section 4.3.

B.2. Authenticated Encryption

An authenticated encryption with associated data (AEAD) scheme considers a keyspace $\mathcal{K}$, a message space $\mathcal{M}$, a ciphertext space $\mathcal{X}$, and a tag space $\mathcal{T}$, and is defined using the following algorithms.

- $k \leftarrow \text{AEAD}.\text{keygen}(1^\lambda)$: Outputs a key $k \leftarrow \mathcal{K}$.
- $(C, \tau) \leftarrow \text{AEAD}.\text{Enc}(k, m; A)$: For a key $k \in \mathcal{K}$, message $m \in \mathcal{M}$, and associated data $A \in \{0, 1\}^*$, outputs a ciphertext $C \in \mathcal{X}$ and a tag $\tau \in \mathcal{T}$.
- $m \vee \perp \leftarrow \text{AEAD}.\text{Dec}(k, C, \tau; A)$: For a key $k \in \mathcal{K}$, ciphertext $C \in \mathcal{M}$, tag $\tau \in \mathcal{T}$, and associated data $A \in \{0, 1\}^*$, outputs a message $m \in \mathcal{M}$ or $\perp$.

Any AEAD scheme must satisfy the following guarantees.

Definition 6 (Correctness). AEAD is correct if and only if the following holds true.

$$\Pr \left[m \leftarrow \text{AEAD}.\text{Dec}(k, C, \tau; A) \mid \begin{array}{l} k \leftarrow \text{AEAD}.\text{keygen}(1^\lambda) \\ (C, \tau) \leftarrow \text{AEAD}.\text{Enc}(k, m; A) \end{array} \right] = 1$$

Definition 7 (Security). An AEAD scheme is secure if it satisfies the IND-CCA notion of security [77].

It is widely known that the AES-GCM block cipher mode of operation satisfies these guarantees [78], where $\mathcal{K} = \{0, 1\}^\lambda$, $\mathcal{M} = \{0, 1\}^*$, $\mathcal{C} = \{0, 1\}^*$. In other words, it can tolerate messages of arbitrary length and produce ciphertexts accordingly.

B.3. Zero-knowledge Signature Verification

We require a scheme for constructing zero-knowledge proofs of knowledge of valid signatures (ZKPVS) of signatures produced during TLS exchanges. Such a scheme considers a prover and a verifier, where the prover holds a valid signature σ issued by a keypair sk, vk , and the verifier holds a list $R = \{vk_i\}_{i \in [n]}$ of all valid public verification keys, where $vk \in R$. Previous work has produced practical schemes for proving knowledge of ECDSA signatures (e.g. see ZKAttest [62] and CDLS [61]), noting their similarity to ring signatures [79], in particular. Similar approaches for other TLS-compliant signature schemes (e.g. based on RSA) exist [80], but do not appear to be practical for our application (though practical constructions would have immediate value for our work).

While some previous work refers to the zero-knowledge functionality that we require as ring signature schemes [60]–[62], we note that the functionality differs in that the eventual proofs are constructed over standard signatures (by non-signing entities). As a result, we give a modified formalisation below in Definition 8 that captures this primitive.

Definition 8 (ZKPoK of Valid signatures). *Let $R = \{vk_i\}_{i \in [n]}$ be a collection of public keys for a valid signature scheme Ψ , and let (sk, vk) be a keypair, such that there exists a $j \in [n]$, where $vk = vk_j$. A scheme for building zero-knowledge proofs of knowledge of valid signatures (ZKPVS), Π , is a tuple of the following algorithms:*

- $\Pi.\text{Prove}(R = \{vk_i\}_{i \in [n]}, \text{Sig}, m)$: *outputs a proof σ of a valid signature Sig with respect to R ;*
- $\Pi.\text{Verify}(R = \{vk_i\}_{i \in [n]}, \sigma, m)$: *outputs a bit $b \in \{0, 1\}$, where $b = 1$ indicates successful verification, and $b = 0$ indicates failure.*

Security properties. We now describe the required security properties of a ZKPVS scheme. Note that the properties bear resemblance with ring signature schemes [79]. First, we say that Π is *complete* if, for any set of keys $\{(sk_i, vk_i)\}_{i \in [n]}$, $j \in [n]$, message m , the set $R = \{vk_i\}_{i \in [n]}$, signature $\text{Sig} \leftarrow \Psi.\text{Sign}(sk_j, m)$, and $\sigma \leftarrow \Pi.\text{Sign}(R, \text{Sig}, m)$, then $1 \leftarrow \Pi.\text{Verify}(R, \sigma, m)$. Second, let **Anon** and **Unf** be the security games defined in Fig. 6. We say that Π is *anonymous* (resp. *unforgeable*) if the advantage of a PPT algorithm, $\mathcal{A}$ in either game is negligible. In both games, the adversary has access to the following oracles:

- $\mathcal{O}_S$: takes as input an index i , a message m' , and a set R' , and returns a proof $\sigma \leftarrow \Pi.\text{Prove}(R', \text{Sig}', m')$ of a valid signature Sig' over m' ;
- $\mathcal{O}_C$: takes as input an index i , and returns the randomness used to generate vk_i .

Furthermore, let $\mathcal{Q}_S$ and $\mathcal{Q}_C$ be the sets of queries sent to $\mathcal{O}_S$ and $\mathcal{O}_C$, respectively.¹⁵

Instantiations. As mentioned above, it is possible to instantiate the required functionality with a specific proof

15. Note that both oracle definitions assume the generation of a global set of key pairs that are used during the security game, and a correspondingly global set, R , of all valid verification keys.

Figure 6. Security games for establishing anonymity and unforgeability guarantees of a ZKPVS scheme Π .

Anon
1 : Let $\{sk_i, vk_i\}_{i \in [n]}$, and $R = \{vk_i\}_{i \in [n]}$
2 : $(m, R, i_0, i_1) \leftarrow \mathcal{A}^{\mathcal{O}_S, \mathcal{O}_C}(\{vk_i\}_{i \in [n]})$
3 : if $[(i_0, i_1 \notin [n]) \vee (vk_{i_0}, vk_{i_1} \notin R)]$: abort
4 : $d \leftarrow \mathcal{A}(\{0, 1\})$
5 : $\text{Sig} \leftarrow \Psi.\text{Sign}(sk_{i_d}, m)$
6 : $\sigma \leftarrow \Pi.\text{Prove}(R, \text{Sig}, m)$
7 : $d' \leftarrow \mathcal{A}(\sigma)$
8 : if $[d' \stackrel{?}{=} d]$: return 1
9 : return 0
Unf
1 : Let $\{sk_i, vk_i\}_{i \in [n]}$, and $R = \{vk_i\}_{i \in [n]}$
2 : $(m^*, R^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_S, \mathcal{O}_C}(R = \{vk_i\}_{i \in [n]})$
3 : if $[(R^* \not\subseteq R) \vee$
4 : $(\exists i' \in \mathcal{Q}_C \text{ s.t. } vk_{i'} \in R^*) \vee$
5 : $(m^* \in \mathcal{Q}_S)]$: abort
6 : return $\Pi.\text{Verify}(R^*, \sigma^*, m^*)$

scheme that generates signatures under ECDSA private keys that preserve anonymity amongst a set of known ECDSA verification keys (e.g. see [60]–[62]). This means that we can directly instantiate our DCTLS protocol for servers using ECDSA signing. Supporting TLS signatures of other types requires practical instantiations of ZKPVS schemes for the specific signing method.

Server anonymity. As noted throughout, we introduce the possibility for $\mathcal{C}$ to prove to $\mathcal{V}$ that the communicating TLS server, $\mathcal{S}$, belongs to a pre-approved set, R , of server identities. This list of identities can be constructed simply from a list of TLS certificates, and uses a compliant ZKPVS scheme (Section B.3) for generating proofs of valid TLS signatures. We note here that anonymity is only preserved amongst the set R , and is highly application-specific: if R only contains a single identity, then anonymity is not guaranteed. However, there are various applications where R is likely to be non-trivial, such as in the case where $\mathcal{C}$ would like to generate a commitment to a bank balance that is in a range. In principle, our approach would allow generating commitments to balances provided by any of a pre-approved list of banks. In doing so, this would preserve $\mathcal{C}$'s privacy by hiding the identity of the bank they have an account with. Similar mechanisms can be built for generating commitments with respect to governmental identities (e.g. social security statuses associated with EU member states). While targeting any given applications are beyond the scope of this work, we believe that the provisioning of the capability for generating such proofs (which previous works do not provide) can provide meaningful privacy enhancements for clients. Note also that a larger R will have an impact on the

performance of the ZKPVS scheme employed, since their complexity is typically dependent on $|R|$. Nonetheless, we believe that even a modest value (*e.g.* $|R| = 10$) would grant meaningful privacy protection. As a concrete example, banking sectors are typically highly consolidated — in the UK, there are four banks that hold the majority of customer accounts [81] — . Providing privacy for this “small” sets can be meaningful for many clients.

Appendix C.

DCTLS Formal Description

The three phases (HSP, QP, CP) of a generic three-party TLS (DCTLS) protocol are formally described (in terms of their inputs and outputs) below.

- $(pp, sp_C, sp_S, sp_V) \leftarrow \text{DCTLS.HSP}(1^\lambda)$: The handshake phase takes as input a security parameter, and computes a TLS handshake between S , and an effective client that consists of both C and V . The public/secret parameters (pp, sp_S) learnt by S are the same as in a standard TLS handshake. The secret parameters learned by C (sp_C) and V (sp_V) are shares of the secret parameters learnt by a standard TLS client [29], so that neither party can compute encrypted traffic alone.
- $(r, \hat{q}, \hat{r}) \leftarrow \text{DCTLS.QP}(pp, sp_C, sp_S, sp_V, q)$: The query phase takes the public and secret parameters of each party as input, along with a query, q , that is to be sent to S . This phase requires S to construct a response, r , to q and return it to C . The phase outputs both q and r , and also vectors of TLS ciphertexts ($\hat{q}$ and $\hat{r}$) that encrypt the client queries and the server responses. $\hat{q}$ and $\hat{r}$ are vectors containing blocks of the TLS ciphertext encrypting q and r , respectively.
- $b \leftarrow \text{DCTLS.CP}(pp, sp_C, sp_V, q, r, \hat{q}, \hat{r}, (i, j))$: The commitment phase outputs a bit b , where $b = 1$ if C constructs a valid opening of $\hat{q}_i$ and $\hat{r}_j$ with respect to the unencrypted q and r . Broadly speaking, C sends to V the TLS-encrypted ciphertexts, before V sends sp_V to C , and then C opens the commitments. Note that a valid opening could be proving in zero-knowledge that $\hat{r}_j$ encrypts a value in a given range, or using 2PC to decrypt the block directly.

Appendix D.

Security of AES-GCM Optimisations

D.1. Secure 2PC Encryption & Decryption

2PC functionalities. We consider the 2PC functionalities for encryption and decryption in 2PC-AES-GCM as given in Algorithm 1 and Algorithm 2, respectively. Our ideal functionality also covers the nonce uniqueness requirement of AES-GCM. We note that in practice these additional checks do not seem to affect the running time by much: for example, our prototype garbled circuit implementation only requires around 768 extra AND gates, representing around a 10% increase over an AES circuit.

Algorithm 1 2PC-AES-GCM Encrypt

Require: $k = k_c + k_v, IV_c, IV_v, \{h_c^i\}_{i \in [n]}, \{h_v^i\}_{i \in [n]}$
Require: C inputs a message M
Require: IV_c and IV_v must not have been supplied for encryption previously.
Ensure: C learns $((C_1, \dots, C_n), \tau(A, C, k, IV))$.
Ensure: V learns $(C_1, \dots, C_n)$.
if $IV_c \neq IV_v$ **then**
 return Error ▷ The IVs must match.
end if
Parse M as $M_1 \parallel \dots \parallel M_n$ ▷ M_i fits AES blocksize
 $C = (C_i \leftarrow \text{AES.Enc}(k_c + k_v, IV_c + i) \oplus M_i)_{i \in [n]}$
 $\tau_c \leftarrow P_{A||C||\text{len}(A)||\text{len}(C)}(\{h_c^i\})$
 $\tau_v \leftarrow P_{A||C||\text{len}(A)||\text{len}(C)}(\{h_v^i\})$
 $\tau \leftarrow \tau_c \oplus \tau_v \oplus \text{AES.Enc}(k_c + k_v, IV_c)$
return (C, τ) to C
return C to V

Algorithm 2 2PC-AES-GCM Decrypt

Require: $k = k_c + k_v, IV_c, IV_v, \{h_c^i\}_{i \in [n]}, \{h_v^i\}_{i \in [n]}$
Require: C inputs a set of n masks $\{b_i\}$ and secret parameters sp for a computationally binding and perfect hiding commitment scheme Γ' , and V inputs the corresponding commitments $\{d_i\}$ that were sent by C , generated using Γ' , and ephemeral challenges $\{t'_i\}$.
Require: C and V jointly input a set of ciphertext blocks $C_1, \dots, C_n$ and a tag $\tau(A, C, k, IV)$.
Require: IV_c and IV_v must not have been supplied for encryption previously.
Ensure: C learns $(M_1, \dots, M_n)$.
Ensure: V learns $(E_1, \dots, E_n)$.
if $\exists i$ s.t. $0 \leftarrow \Gamma'.\text{Open}(sp', d_i, t'_i, b_i)$
 return Error ▷ Commitment checks failed.
end if then
if $IV_c \neq IV_v$ **then**
 return Error ▷ The IVs must match.
end if
 $\tau'_c \leftarrow P_{A||C||\text{len}(A)||\text{len}(C)}(\{h_c^i\})$
 $\tau'_v \leftarrow P_{A||C||\text{len}(A)||\text{len}(C)}(\{h_v^i\})$
 $\tau' = \tau'_c \oplus \tau'_v \oplus \text{AES.Enc}(k_c + k_v, IV_c)$
if $\tau' \neq \tau$ **then**
 return Error ▷ Invalid tag
end if
 $(M_i = C_i \oplus \text{AES.Enc}(k_c + k_v, IV_c + i))_{i \in [n]}$
 $(E_i = \text{AES.Enc}(k_c + k_v, IV_c + i) \oplus b_i)_{i \in [n]}$
return $(M_i)_{i \in [n]}$ to C
return $(E_i)_{i \in [n]}$ to V

Security argument. We now argue the security of computing encryptions and decryptions with respect to the ideal functionalities described in Algorithm 6 and Algorithm 7. We implicitly assume that 2PC evaluations of the polynomial P and the AES functionality (using garbled circuits) are secure with respect to malicious adversaries, and that AES-GCM is a secure AEAD scheme. These security guarantees

are assumed in previous work [2], [17], [18], but are not made explicit. We require them when proving that the query phase of DiStefano is secure (Section E.2).

Lemma 9 (Malicious Client). *2PC-AES-GCM is secure in the presence of a malicious adversary that controls $\mathcal{C}$.*

Proof. Let S be a PPT simulator for the encryption functionality, that simply returns samples C' from the domain of AES.Enc , and $\tau_c \leftarrow \$ \{0, 1\}^t$, and returns (C, τ_c) to $\mathcal{C}$. We ultimately argue that the real-world outputs of 2PC-AES-GCM are indistinguishable from this.

Let S_{AES} be a simulator for the ideal 2PC evaluation of AES.Enc , and let S_P be a simulator for the ideal evaluation of P . It first sends m to S_{AES} and learns $C = (C_1, \dots, C_n)$. Then it sends C to S_P (along with A) and learns τ . It returns (C, τ) to $\mathcal{C}$. To see that this is indistinguishable from the real-world, we can trivially construct a hybrid argument from the real-world protocol that relies on two steps, replacing real garbled circuit evaluation of each functionality with ideal-world simulation, and argue security based on the maliciously-secure 2PC garbled circuit approach that we use (Section 3).

Finally, based on the assumption that AES is a pseudo-random permutation, we can construct a final hybrid step, that replaces AES.Enc with a random value in the domain.¹⁶

The case of decryption is much simpler since the client only learns the message if they submit valid inputs to S (by the AEAD security guarantees of AES-GCM). This can be established using the same simulators S_{AES} and S_P defined above. $\square$

Lemma 10 (Malicious Verifier). *2PC-AES-GCM is secure in the presence of a malicious adversary that controls $\mathcal{V}$.*

Proof. The proof for a malicious $\mathcal{V}$ follows the same structure as in the case of $\mathcal{C}$, but note that $\mathcal{V}$ is strictly less powerful, because the $\mathcal{V}$ does not submit a message to be encrypted. $\square$

We briefly note that PageSigner follows a slightly different approach than this for computing tags: we decided not to follow their approach, as a “back-of-an-envelope” calculation suggests that it is strictly slower than the aforementioned approach. We discuss this in more detail in Appendix F.

D.2. Commitment Scheme

We prove that the commitment scheme for AES-GCM ciphertexts presented in Section 5.1 is a perfectly hiding and computationally binding commitment scheme. Concretely, this means showing that the 2PC’s Algorithm 2 produces computationally binding and perfectly hiding commitments E_i to $e_i = \text{AES.Enc}(k, IV + i)$. Since the primary application of this work is to prove facts about traffic received from the server, we focus on only the AES Decrypt algorithm. If we wanted to prove facts about client-encrypted traffic,

Algorithm 1 would have to be updated to also include commitments. As before, we implicitly assume that the 2PC evaluations of the polynomial P and the AES functionality are secure with respect to malicious adversaries.

First, let Γ' be a perfectly hiding, and computationally binding commitment scheme, generating commitments $d \in \{0, 1\}^\lambda$ for arbitrary $x \in \{0, 1\}^*$. Let $\mathcal{K}$ and $\mathcal{X}$ be the key and ciphertext spaces for AES-GCM, respectively. Then, let $\text{sp}' \leftarrow \Gamma'.\text{Gen}(1^\lambda)$, and $d_i = \Gamma'.\text{Commit}(\text{sp}', b_i)$ for some $b_i \leftarrow \$ \mathcal{X}$.

Lemma 11 (AES-GCM Commitment security). *The algorithm 2PC-AES-GCM Decrypt, when instantiated with Γ' , produces computationally binding and perfectly hiding commitments to decryptions of AES-GCM ciphertexts.*

Proof. We first formally describe the AES-GCM commitment (Γ_{AES}) scheme for any of the i^{th} message blocks, using the following functionality (applying the framework described in Section B.1).

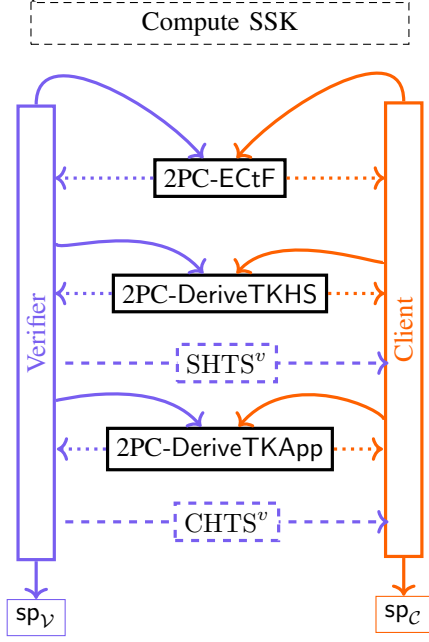
- $\text{sp} = (b_i, k_c)$, and assume that the receiver holds the commitment $d_i \leftarrow \Gamma'.\text{Commit}(\text{sp}', b_i)$.
- $E_i \leftarrow \Gamma_{\text{AES}}.\text{Commit}(\text{sp}, M_i)$: Runs the 2PC-AES-GCM-Commit algorithm to generate commitments $(C_i, E_i = e_i + b_i)$, where C_i is the i^{th} received ciphertext, and $e_i = \text{AES.Enc}(k_c + k_v, IV_c + i)$.
- $(k_v, t'_i) \leftarrow \Gamma_{\text{AES}}.\text{Challenge}(C_i, E_i)$: reveals $\mathcal{V}$ ’s key share, k_v , to the commitment sender, along with a challenge t'_i for Γ' .
- $\hat{b} \leftarrow \$ \Gamma_{\text{AES}}.\text{Open}(\text{sp}, (C_i, E_i), k_v, M_i)$: First, checks that $1 \leftarrow \Gamma'.\text{Open}(\text{sp}', d_i, t'_i, b_i)$. Then, computes $b_i \oplus E_i$ to reveal e_i , and then returns 1 iff $M' \leftarrow C_i \oplus e_i$ satisfies $M' = M_i$.

To argue perfect hiding, notice that b_i is not revealed to $\mathcal{V}$ during the execution of 2PC-AES-GCM-Commit. As Γ' is a perfectly hiding commitment scheme, we may simply replace b_i with a uniformly random value r_i in the range of b_i , which in turn makes $E_i = r_i \oplus e_i$ a one-time pad encryption of e_i . By the properties of the one-time pad, we have that the scheme is therefore also a perfectly hiding commitment to $e_i = \text{AES.Enc}(k_c + k_v, IV + i)$.

To argue computational binding, we first ensure that the masks b_i generated by the client are consistent with their input to the 2PC-AES-GCM-Commit algorithm by explicitly checking that they correspond to the verifier-known commitments. To argue security henceforth, we consider two possible events. In the first event, we consider a PPT adversary $\mathcal{B}'$ that can generate valid openings of d_i to $b' \neq b_i$ for Γ' . The advantage of $\mathcal{B}'$ is clearly bounded by the computational binding security of Γ' . In the second event, we assume that no such $\mathcal{B}'$ exists, and instead we consider a PPT adversary $\mathcal{B}$ that finds M' , such that $M' = E_i \oplus b_i \oplus C_i$ for $M' \neq M_i$. Since the only free variable in this equation is $e_i = \text{AES.Enc}(k_c + k_v, IV_c + i)$, this would require $\mathcal{B}$ to find $k' \neq k_c \oplus k_v$ such that $\text{AES.Enc}(k', IV_c + i) = e_i$. Clearly, by the IND-CCA security of AES-GCM, finding such a $k' \in \mathcal{K}$ is computationally infeasible. Note that the lack of key-committing security does not play a role here:

16. This only holds if the IV is a nonce, see [2, §B.2].

Figure 7. Ordered execution of 2PC exchange between $\mathcal{C}$ and the $\mathcal{V}$ during the handshake phase of DiStefano.



the adversary would need to freely manipulate the value of e_i to launch such an attack, which is impossible under the assumption that b_i is fixed. $\square$

Appendix E.

DCTLS Standalone Security Model

E.1. Handshake Phase Security

For establishing the security of the handshake phase, we need to show that $\mathcal{C}$ (in cooperation with $\mathcal{V}$) and $\mathcal{S}$ establish a secure TLS 1.3 channel. To do this, we leverage the handshake phase security analysis of Oblivious TLS [17, Section 6], that is built upon the multi-stage key exchange model used to prove the standard handshake phase TLS 1.3 protocol [29]. The analysis of [29] follows the Bellare-Rogaway (BR) framework [82] for establishing authenticated key exchange security based on session key indistinguishability, and builds on the multistage model of Fischlin and Günther [?, [83]. In summary, this model considers an adversary that: interacts with several concurrent TLS 1.3 sessions between different endpoints (each of which has its own identifier); can intercept, drop, and inject messages between entities; can corrupt endpoints to learn their secret parameters; and can request specific leakage of established keys. The two core security properties that an adversary is attempting to break are known as *Match Security*¹⁷ and *Multi-Stage Security*.¹⁸

17. That any two sessions with identical identifiers will agree on the same key eventually.

18. That any tested key is indistinguishable from a random string of the same length.

Algorithm 3 2PC-ECtF ideal functionality

Require: $ssk_c = Y_s^{x_c}$, $ssk_v = Y_s^{z_v}$

Ensure: Output shares t_c to $\mathcal{C}$, and t_v to $\mathcal{V}$ of the x -coordinate of $Z = Y_s^{x_c + z_v}$

Algorithm 4 2PC-DeriveTKHS ideal functionality

Require: (t_c, H_0, H_1) from $\mathcal{C}$

Require: (t_v) from $\mathcal{V}$

Ensure: For each $w \in \{c, v\}$: **return** $(HS^w, CHTS^w, SHTS^w, dHS^w, tk_{chs}^w, tk_{shs}^w)$ to $\{\mathcal{C}, \mathcal{V}\}$

To prove the above security properties, the work of [17] models $\mathcal{C}$ as a multi-party entity known as a *TLS engine*. The differences in comparison with the standard model of [29] are: (1) the handshake traffic keys are leaked to the multi-stage adversary *only* when $\mathcal{C}$ is corrupted; (2) the MAC keys used in CF and SF messages are leaked to the multi-stage adversary upon reception of the corresponding messages; (3) the IVs are leaked to the adversary; (4) the adversary has the ability to make the engines abort; (5) the adversary is able to shift the computed secret by an arbitrary scalar Q_ϵ .

For DiStefano, one crucial difference in our approach from the TLS engine model of [17] is in criterion (1): we only reveal handshake traffic keys when the *client* is corrupted, and not when the verifier is. It's worth recalling that criterion (5) is permitted (as it is in [17]) since $\mathcal{V}$ can arbitrarily influence the session secret by scalar multiplication. This means that the security of DiStefano is likewise based on the *Shifted PRF ODH assumption* [84]. See [17, Definition 2] for more details. We also require (as in [17]) the additional property that the adversary can only test handshake keys if both $\mathcal{C}$ and $\mathcal{V}$ of a connection are completely honest. Finally, we only allow the adversary to corrupt a single party within any given session.

To summarise, the DiStefano security model essentially provides the adversary with a subset of the capabilities of the adversary in [17]. Note that a potential strengthening of the security model could include the adversary learning $\mathcal{S}$'s identity when it corrupts $\mathcal{C}$. However, such information only becomes pertinent during the commitment phase, when we later consider the case of a malicious $\mathcal{V}$. Since we only allow corruption of a single entity in a single session, we do not consider this possibility during the handshake phase of the protocol.

Applying this model to DiStefano. To use the model defined above, we analyse the 2PC interaction between $\mathcal{C}$ and $\mathcal{V}$, and show that a corrupted client/verifier can only learn details linked to criteria (1)–(5) above. Fig. 7 gives a summary of the 2PC interactions between $\mathcal{C}$ and $\mathcal{V}$, where Algorithm 3, Algorithm 4, and Algorithm 5 give descriptions of the ideal 2PC functionalities that are used.¹⁹ Our proof is situated in the standard model.

19. See Fig. 2 for the full TLS derivation of each value.

Algorithm 5 2PC-DeriveTKApp ideal functionality

Require: (dHS_c) from $\mathcal{C}$ **Require:** (dHS_v) from $\mathcal{V}$ **Ensure:** For $w \in \{c, v\}$: **return** $(\text{tk}_{capp}^w, \text{tk}_{sapp}^w)$ to $\{\mathcal{C}, \mathcal{V}\}$

Before any 2PC takes place, the client and the verifier compute a shared value $\text{SSK} = g^{x_c + z_v}$, where x_c and z_v are the secrets of the respective participants. In this portion of the execution, it is possible for either participant to *shift* the session key by a certain scalar value, taken from the scalar field associated with the group that is being used. Criterion (5) captures this capability for an adversary, by allowing them to shift the eventual shared secret by a scalar value once they have corrupted one of the participants.

In each executed 2PC functionality, $\mathcal{C}$ and $\mathcal{V}$ can control their inputs to each function, and produce a value that is used in subsequent stages of the TLS protocol. By using maliciously-secure 2PC garbled circuit protocols, we reduce the “cheat-down” ability for either party to breaking any of the individual primitives executed within the garbled circuits. Fortunately, each of these primitives is already proven secure individually, and in a non-2PC TLS setting [29]. In other words, using these primitives does not permit any additional capabilities to an adversary that corrupts either party.

Therefore, criteria (1)-(4) are explained in the following. As noted in [17], $\mathcal{V}$ must reveal certain values (SHTS^v and CHTS^v) to allow $\mathcal{C}$ to decrypt handshake traffic before the application session keys are derived. As was shown in [29], revealing this information after committing to server-encrypted ciphertexts is safe, since the eventual application traffic secrets are independent of the handshake-encryption traffic keys. This protects against a malicious client, but means that any adversary that corrupts $\mathcal{C}$ learns all of the intermediate secrets that are used for encrypting and decrypting traffic *during* the handshake. On the other hand, a malicious verifier sending incorrect values will immediately be discovered since $\mathcal{C}$ will no longer be able to decrypt any traffic.

We can now finalise the security of the handshake into the following theorem.

Theorem 12 (Security of handshake phase). *The DiStefano protocol is secure with respect to the ideal handshake phase functionality (DCTLS.HSP), when assuming the following:*

- a maliciously-secure 2PC-ECtF protocol;
- a maliciously-secure 2PC-DeriveTKHS protocol;
- a maliciously-secure 2PC-DeriveTKApp protocol;
- the hardness of the Shifted PRF ODH problem [17, Definition 2];
- the underlying security of the TLS 1.3 protocol [29].

The proof of this theorem follows a standard hybrid argument, where at each stage the 2PC protocol is replaced with an ideal functionality that computes the same result. Since each 2PC protocol is executed in sequence, this proof argument follows in the standard model. Once the ideal

Algorithm 6 2PC-RL-Encrypt ideal functionality

Require: $(\text{tk}_{capp}^c, q, AD)$ from $\mathcal{C}$ **Require:** (tk_{capp}^v, AD) from $\mathcal{V}$ $(\hat{q}, \tau_{\hat{q}}) \leftarrow \text{AEAD}.\text{Enc}(\text{tk}_{capp}, q; AD)$ **return** Output $(\hat{q}, \tau_{\hat{q}})$ to $\mathcal{C}$ **return** Output $\hat{q}$ to $\mathcal{V}$

Algorithm 7 2PC-RL-Decrypt ideal functionality

Require: $(\text{tk}_{sapp}^c, (\hat{r}, \tau_{\hat{r}}), AD)$ from $\mathcal{C}$ **Require:** (tk_{sapp}^v) from $\mathcal{V}$ **return** $\text{AEAD}.\text{Dec}(\text{tk}_{sapp}, \hat{r}, \tau_{\hat{r}}; AD)$ to $\mathcal{C}$

functionality is used, the rest of the security proof follows from the same properties that guarantee security of the underlying TLS 1.3 handshake protocol. A very similar security proof was given in [17] in the universal composability framework.

As a consequence, the security of DiStefano is confirmed, based on the choices of 2PC protocols that are used. The MPC primitives that we use and implement satisfy malicious security, and are discussed formally in Section 3 and Section 4. Our experimental results in Section 7 detail how performance changes depending on the choice of 2PC primitives.

E.2. Query Phase Security

As in the handshake phase, while the server is left untouched, we continue to consider the client and the verifier as one that works together to encrypt and decrypt packets to and from $\mathcal{S}$. This is a requirement, since the end of the handshake phase of a DCTLS protocol leaves the client and verifier with shares of the secret session parameters, that need to be combined in order to construct messages.

In effect, the query execution phase considers two ideal functionalities: 2PC-RL-Encrypt (Algorithm 6), and 2PC-RL-Decrypt (Algorithm 7). In 2PC-RL-Encrypt, the client and the verifier submit their secret parameters, and the client submits a query (e.g. an HTTP request). The ideal functionality returns an encryption of this query, under a TLS 1.3-compliant AEAD scheme (Section B.2). In 2PC-RL-Decrypt, the client and the verifier submit the same inputs, and the client submits a ciphertext received from the server, and the ideal functionality returns the decryption of this ciphertext under the same AEAD scheme, or $\perp$ in the event that the ciphertext does not decrypt properly.

We can show that the query phase of DiStefano is secure when $\text{AEAD} = \text{AES-GCM}$, assuming the security of the 2PC-AES-GCM protocol (Section D.1). The proof that the query phase of DiStefano satisfies security with respect to the ideal DCTLS.QP functionality follows once we have protocols that are secure with respect to 2PC-RL-Encrypt and 2PC-RL-Decrypt. The proof that 2PC-AES-GCM satisfies both follows almost immediately from Lemma 9 and Lemma 10, due to the similarity between the ideal functionality for 2PC-RL-Encrypt (2PC-RL-Decrypt) and 2PC-AES-

GCM Encrypt (2PC-AES-GCM Decrypt). We state the full theorem below for completeness.

Theorem 13. *The DiStefano protocol is secure with respect to the ideal query phase functionality (DCTLS.QP), when assuming a maliciously-secure 2PC-AES-GCM protocol, and the underlying security of the TLS 1.3 protocol [29].*

E.3. Commitment Phase Security

For the commitment phase of DiStefano, we split the requirement into a number of sub-properties: session privacy (SPriv), ring session authentication (SAuth_n¹), and session unforgeability (SUnf). In each model, we first assume that secure handshake and query phases have been computed, using the ideal functionalities (DCTLS.HSP, DCTLS.QP) (Appendix C). Recall that we only consider adversarial corruption of a single party in any situation, and, therefore, for any post-handshake security game, we consider only handshake phase corruptions concerning the same party.

In each of the security models (Fig. 8), we consider a (potentially dishonest) $\mathcal{C}$ that starts by sending a commitment, $c_{\hat{q},\hat{r}}$, to a specific session, S . In SPriv, the honest client $\mathcal{C}$ constructs and sends a proof, $\sigma_{S,\text{sid},L}$, that $c_{\hat{q},\hat{r}}$ is a commitment to a TLS session established with $S \in L$ (L is the set of accepted servers). The adversarial verifier, $\mathcal{A}_V$, succeeds if it identifies the identity of S (it can point which server in the set L that $\mathcal{C}$ is communicating with). In SAuth_n¹, we consider an adversarial client, $\mathcal{A}_C$, where the communication in the security game is the same, except that $\mathcal{A}_C$ succeeds if the commitment $c_{\hat{q},\hat{r}}$ corresponds to a session S established with a server $S' \notin L$. Finally, in SUnf, $\mathcal{V}$ reveals their secret session data to $\mathcal{A}_C$, and $\mathcal{A}_C$ succeeds if it can open $c_{\hat{q},\hat{r}}$ to a different session S' . Overall, we show that each of the properties follows, assuming a sufficiently binding and hiding commitment scheme, and a ZKPVS scheme for TLS certificates for showing that $S \in L$ (e.g. [60], see Section B.3).

Session privacy. For protecting the privacy of sessions during the commitment phase, i.e. that the client commitment does not reveal any information about the session to a malicious $\mathcal{V}$, we show that DiStefano satisfies security in the SPriv security game (Fig. 8).

Lemma 14. *Let Γ be a computationally hiding commitment scheme for a DCTLS scheme, and let Π be a ZKPVS scheme that satisfies anonymity for TLS certificates. Then, for all PPT algorithms $\mathcal{A}$, we have that:*

$$\text{Adv}_{\mathcal{A}, \text{DCTLS}, \Gamma}^{\text{SPriv}}(\lambda) < \text{negl}(\lambda).$$

Proof. We construct our proof of security as a two-step hybrid proof. In the first step, Π is modified to always sign using the secret key of server S^0 , regardless of the bit d . In the second step, the commitment scheme is modified to always commit to traffic exchanged with S^0 , regardless of the choice of bit d . We can see that steps above can be

arbitrarily changed to always commit to traffic exchanged with S^1 , therefore, we will speak only about the S^0 , without loss of generality.

Note that once both hybrid steps have been executed, the adversary $\mathcal{A}_V$ has no advantage in guessing the bit d , since they always receive session commitments and zero-knowledge proofs of valid signatures for the traffic received from a single server. Therefore, we simply have to show that the real execution of SPriv is indistinguishable from this case to show that DCTLS satisfies SPriv security.

The distinguishing probability between the two views in the first hybrid step can be bounded by the anonymity property of Π . In other words, if there is an adversary $\mathcal{A}$ that distinguish between the two steps, then there is an adversary $\mathcal{B}$ that can break the Anon security game of Π (Fig. 6). This follows since, in the case when $d = 1$ the only difference is the fact that σ is always computed over the certificate of S^0 . Therefore, $\mathcal{B}$ can simply forward the message to be signed during the TLS execution to their challenger, and receive back the signature σ . Then, they can send this signature back to $\mathcal{A}$ and output whatever $\mathcal{A}$ outputs. If $\mathcal{A}$ has non-negligible advantage in distinguishing between the two steps, then so will $\mathcal{B}$.

The distinguishing probability between the two views in the second hybrid step can be bounded by the fact that the session commitment is generated only for S^0 . As such, any adversary $\mathcal{B}$ against the computational hiding property of Γ forward their challenge commitment to $\mathcal{A}$ in the same as before, and win with the same advantage as $\mathcal{A}$. This completes the proof. $\square$

Ring authentication. We show that DiStefano ensures that a malicious $\mathcal{C}$ must authenticate S to $\mathcal{V}$, out of a set L possible n accepted servers (where L is specified by $\mathcal{V}$) using the SAuth_n¹ security game (Fig. 8).

Lemma 15. *Let Π be a ZKPVS scheme that satisfies unforgeability for TLS certificates. Then, for all PPT algorithms $\mathcal{A}$, we have that:*

$$\text{Adv}_{\mathcal{A}, \text{DCTLS}, \Pi}^{\text{SAuth}_n^1}(\lambda) < \text{negl}(\lambda).$$

Proof. Suppose that $\mathcal{C}$ could win the SAuth_n¹ game with non-negligible advantage. Then, an adversary $\mathcal{B}$ that is attempting to forge proofs for Π can simply output whatever signature $\mathcal{A}_C$ outputs as their answer to the Unf security game (Fig. 6). If $\mathcal{A}_C$ creates a valid forgery, then the unforgeability of Π is violated. $\square$

Session unforgeability. We show that a malicious $\mathcal{C}$ cannot open commitments to sessions that were not previously committed to, by showing that DiStefano satisfies security in the SUnf security game (Fig. 8).

Lemma 16. *Let Γ be a perfectly binding commitment scheme for a DCTLS scheme. Then, for all PPT algorithms $\mathcal{A}$, we have that:*

$$\text{Adv}_{\mathcal{A}, \text{DCTLS}, \Gamma}^{\text{SPriv}}(\lambda) < \text{negl}(\lambda).$$

Figure 8. Security games for the commitment protocol.

SPriv	SAuth_n¹
1 : $L \leftarrow \mathcal{A}_V(1^\lambda, 1^N)$	1 : $L \leftarrow \mathcal{V}(1^\lambda, 1^N)$
2 : $(sk_\Pi, vk_\Pi) \leftarrow \Pi.setup(1^\lambda)$	2 : $(sk_\Pi, vk_\Pi) \leftarrow \Pi.setup(1^\lambda)$
3 : $(pp^0, sp_C^0, sp_S^0, sp_V^0) \leftarrow DCTLS.HSP(1^\lambda, \mathcal{C}, \mathcal{S}^0, \mathcal{A}_V)$	3 : $(pp, sp_C, \perp, sp_V) \leftarrow DCTLS.HSP(1^\lambda, \mathcal{A}_C, \mathcal{S}, \mathcal{V})$
4 : $(pp^1, sp_C^1, sp_S^1, sp_V^1) \leftarrow DCTLS.HSP(1^\lambda, \mathcal{C}, \mathcal{S}^1, \mathcal{A}_V)$	4 : if $[S \in L]$: return 0
5 : if $[(S^0 \notin L) \vee (S^1 \notin L)]$: return 0	5 : $\sigma \leftarrow \mathcal{A}_C(pp, sp_C, sk_\Pi, vk_\Pi, L)$
6 : $d \leftarrow \{0, 1\}$	6 : return $\Pi.Verify(vk_\Pi, pp, sp_V, \sigma, L)$
7 : $q \leftarrow \mathcal{A}_V(L)$	SUnf
8 : $(\hat{q}, \hat{r}) \leftarrow DCTLS.QP(pp^d, sp_C^d, sp_S^d, sp_V^d, q)$	1 : $(pp, sp_C, sp_S, sp_V) \leftarrow DCTLS.HSP(1^\lambda, \mathcal{A}_C, \mathcal{S}, \mathcal{V})$
9 : $\sigma \leftarrow \Pi.Sign(sk_\Pi, pp^d, sp_C^d, L)$	2 : $q \leftarrow \mathcal{A}_C(1^\lambda, pp, sp_C)$
10 : $c^d \leftarrow \Gamma.Commit(sp_C^d, \hat{q}, \hat{r})$	3 : $(\hat{q}, \hat{r}) \leftarrow DCTLS.QP(pp, sp_C, sp_S, sp_V, q)$
11 : $d' \leftarrow \mathcal{A}_V(\{pp^d, sp_V^d\}_{d \in \{0,1\}}, vk_\Pi, q, c^d, \sigma, L)$	4 : $c \leftarrow \mathcal{A}_C(sp_C, \hat{q}, \hat{r})$
12 : if $[d' \stackrel{?}{=} d]$: return 1	5 : $q' \leftarrow \mathcal{A}_C(sp_C, sp_V, q, \hat{q}, \hat{r}, c)$
13 : return 0	6 : if $[(\Gamma.Open(sp_V, q', c)) \wedge (q' \neq q)]$: return 1
	7 : return 0

Proof. It is clear to see that an adversary attempting to break the perfect binding property of Γ can utilise the adversary $\mathcal{A}_V$ against SUnf to establish a valid opening based on an uncommitted value. $\square$

Appendix F. PageSigner and AES-GCM

In this section, we compare our approach to computing AES-GCM tags to the approach employed by PageSigner [12]. In a 2PC setting, we assume that both k and the powers of $h = h_c + h_v$ are additively shared by both parties, with C , IV and A acting as public inputs.

Assuming that C is a single block without any associated data (i.e. $C = C_1$), we have $\tau = (h_c^{m-2} + h_v^{m-2}) \cdot (h_c^1 + h_v^1) \cdot C_1 = (h_c^{m-1} + h_v^{m-1} + h_c^{m-2} \cdot h_v^1 + h_v^{m-2} \cdot h_c^1) \cdot C_1$. As the first of these terms can be computed locally, the cost of computing τ can be reduced to computing $(h_v^{m-2} \cdot h_c + h_c^{m-2} \cdot h_v) \cdot C_1$ in 2PC. This approach can actually be written as a variant of our approach, as the left hand-side is fixed for a particular sharing of h . However, PageSigner instead repeats this process each time a tag is computed. Interestingly, it turns out that simply computing a sharing of $h_v^2 h_c$ and $h_c^2 h_v$ is sufficient to tag blocks of arbitrary length, lowering the cost of tagging to just two OT-based multiplications.

From a performance perspective, a “back-of-an-envelope” calculation shows that this approach is strictly less efficient than the one that we adopt in Section 5. Intuitively, this is because our approach allows all polynomial evaluation to be done locally, even while both approaches require computing an initial sharing of h and its powers, PageSigner’s approach explicitly requires computing two

OT-based multiplications per tagging. Concretely, instantiating these multiplications using the maliciously-secure scheme presented in [56] with 128-bits of statistical security would require 2048 oblivious transfers of 128-bits for the multiplication alone, requiring around 32KiB of bandwidth per tag. In contrast, our scheme only requires transferring around 64 bytes per tagging operation. In other words, our scheme requires around $500\times$ less bandwidth per tagging operation than the approach employed by PageSigner.

Appendix G. Proofs Over Encrypted Data

DiStefano can be used to provide statements in zero knowledge about encrypted data transmitted during a TLS 1.3 session. Specifically, it can provide proofs that a specific substring appears on said data which, in turn means, that the confidentiality of the data remains and only what is needed is revealed.

Revealing a substring. We briefly show how DiStefano can implement two specific optimisations presented by DECO: “Selective Opening”, which allows $\mathcal{C}$ to reveal that a certain substring is present in a plaintext M , and “Selective Redacting”, which allows $\mathcal{C}$ to reveal the entirety of M , other than some selection of omitted substrings.

Using our AES-GCM protocol, both approaches are easily achievable. Suppose that $\mathcal{C}$ is committing to some set of ciphertexts $C_1, \dots, C_n$ for the purpose of proving a statement. Since $\mathcal{C}$ is required to commit to their additive shares of the decryption keys k_i^c before learning V ’s key shares, selectively opening C_i simply requires revealing k_i^c to $\mathcal{V}$. Similarly, $\mathcal{C}$ can selectively reveal any combination of ciphertexts by simply revealing those individual keys. In practice, revealing each block is rather cheap, requiring only 128-bits of bandwidth. In addition, this scheme can

be adapted to deal with substrings inside a single block C : rather than revealing k_i^c directly, C and V instead decrypt C in a garbled circuit with the output masked by ρ that is chosen by C . We remark that this approach is somewhat fragile: for any soundness to hold, we would also require that C is only allowed to modify certain portions of the output plaintext. We view this difficulty as orthogonal to this work: this would require more extensive zero-knowledge proofs.

Combining with server anonymity. Note that substring revealing procedures may be at odds with the anonymity property provided by the ZKPVS scheme. For instance, if we take the example of proving a sufficient bank balance, then different banks may encode the account balance in different ciphertext blocks. Therefore, while the ZKPVS scheme may hide which bank is used by the client, the selective opening process may inadvertently leak the identity of the bank by opening a specific ciphertext block. Any implementation of attestations based on the DiStefano framework must be cognizant of these discussions.

Appendix H. Incompatibility With Garble-then-Prove

In this section we highlight why we have not incorporated the recent improvements presented in [14] into our work. We stress that there are no fundamental incompatibilities between DiStefano and the improvements made by [14]; instead, the difficulties are solely implementation driven. We consider producing a tool that allows us to incorporate these changes to be a pressing, but orthogonal, open problem.

An optimised 2PC-HMAC. We briefly recall the secret derivation optimisation presented in [14]. For the purposes of exposition we shall first show how to efficiently compute the HMAC function in a 2PC setting before incorporating this into TLS1.3 secret derivation.

Let $H = \text{SHA256}$. Recall that the $\text{HMAC}_{\text{SHA256}}$ of a variable length message m with a key k is

$$\text{HMAC}_H(k, m) = H((k \oplus \text{opad}) || H((k \oplus \text{ipad}) || m)).$$

From this we can see that naively computing HMAC_H in 2PC is likely to be expensive, as any generic circuit would be required to compute the hash of m in 2PC. Given that the cost of computing such a hash is proportional to the length of m , such a circuit would likely perform poorly on long messages.

This situation was considered in the context of CBC-HMAC by [2]. In this setting, $\mathcal{P}$ and $\mathcal{V}$ wish to compute the CBC-HMAC of a message m that is known entirely by $\mathcal{P}$ under a shared key k . In order to implement this functionality efficiently, the authors of [2] take advantage of the underlying structure of SHA256. Namely, suppose that m_1 and m_2 are two correctly sized blocks. Then

$$\text{SHA256}(m_1, m_2) = f_H(f_H(IV, m_1), m_2)$$

where IV is the initialisation vector and f_H is the compression function of SHA256. If we now return to the HMAC computation, it is clear that the inner-most call is $f_H(IV, k \oplus \text{ipad})$. Given that f_H is assumed to be a one way function, revealing $s_0 = f_H(IV, k \oplus \text{ipad})$ does not reveal anything about k to either party; thus, we can realise HMAC more efficiently by simply revealing s_0 to $\mathcal{P}$, allowing them to compute the hash of m locally i.e outside of 2PC. This means that the HMAC computation of an arbitrarily long message only requires a few SHA256 calls in 2PC, rather than potentially many when realising HMAC generically.

This idea was recently adapted to the context of TLS 1.2 secret derivation by the authors of [14]. Briefly, the authors propose revealing the value $s_0 = f_H(IV_0, \text{pms} \oplus \text{ipad})$ to both parties, allowing some of the f_H calls to be realised locally. In addition, the authors of [14] also propose re-using previously garbled values across multiple circuits, allowing even fewer f_H calls to be carried out in 2PC. Concretely, this optimisation reduces the number of needed f_H calls in 2PC from 18 to only 6.

Implementation difficulties. We now highlight why this approach seems difficult to realise in DiStefano. At a high-level, the main issue is that our current garbled circuit library (**emp**) does not support either accepting new input or outputting partial values as the circuit runs. Indeed, if **emp** were to support this feature, then realising the optimisation presented would be easy. However, the fact that **emp** does not support this feature means that we would have to realise this optimisation by chaining multiple circuits together. In this model, each circuit computes a sub-portion of the secret derivation procedure and outputs some intermediate results to each party.

On the one hand, this approach would allow each party to locally compute some of the calls to f_H , yielding the claimed speed-ups. However, this would require each circuit to either recompute any shared input, and to validate that the same value of s_0 is used across multiple circuits. In the case of the former, we would need to recompute pms in each individual circuit, and we would also need to check that the same value of s_0 is supplied by both parties. Put simply, this decomposition would reduce some of the potential speed-ups from [14]. Moreover, decomposing secret derivation into multiple circuits would allow a malicious party to alter their inputs at each stage, potentially causing selective failure. Whilst we stress that we cannot see an obvious way to use this to expose a vulnerability in the security of DiStefano, it would contradict elements of the security model that we consider.

Nonetheless, none of these reasons invalidate the approach taken in [14], and we believe that a similar idea can be applied even with the current version of **emp**. For example, notice that our secret derivation circuit recomputes the compression function $f_H(\text{HS} \oplus \text{ipad})$ a total of three times during secret derivation; re-using this value inside the same circuit would be more efficient than our current approach. A similar approach can also be applied to the derivation of traffic secrets. However, we have not implemented this

approach, as we believe that it is unlikely to be competitive with [14] in practice. Put differently, we believe that the completion of a tool that allows for the [14] optimisation to be realised securely inside DiStefano should be the priority for future work.

Appendix I.

Potential Attacks on Janus and DECO’s TLS 1.3 Variant

In this section we present an attack on an *interpretation* of the TLS 1.3 variant of DECO and the recently presented Janus [15] TLS attestation mechanism. Notably, this attack allows either a malicious client to produce false attestations, or for a malicious proxy to fully decrypt all traffic. We stress that our interpretation of DECO’s TLS 1.3 variant may be different from exactly what the authors intended, due to lack of specificity, and thus the attack presented may not be directly applicable. However, the attack presented here applies to any TLS attestation protocol that does not carefully handle the sharing (and disclosure) of traffic secrets.

The SF message. The attack presented in this section exploits a flaw in how both Janus and DECO’s TLS 1.3 variant handle session authentication between the client and the verifier. In order to explain this flaw, we briefly recall the TLS 1.3 server authentication mechanism used in TLS 1.3 (see [29] for a more thorough explanation).

First, assume that $\mathcal{S}$ and $\mathcal{C}$ are establishing a TLS 1.3 session; and that $\mathcal{S}$ and $\mathcal{C}$ have exchanged both the hello messages (ClientHello and CH) and derived authentic copies of the handshake secrets SHTS and CHTS. At this stage, $\mathcal{S}$ authenticates itself to $\mathcal{C}$ as follows:

- 1) To ensure authenticity, $\mathcal{S}$ sends its certificate SCRT to $\mathcal{C}$, followed by a signature on the hash of the session transcript (SCV). Upon receipt of these messages, $\mathcal{C}$ checks the certificate and uses the corresponding public key to verify SCV. Notably, $\mathcal{C}$ is able to check SCV because $\mathcal{C}$ has a full view of the transcript.
- 2) To ensure integrity, $\mathcal{S}$ uses SHTS to derive the *finished* key fk_s . $\mathcal{S}$ then computes the *server finished* (SF) message, by computing a HMAC on the hash of the session transcript using fk_s . $\mathcal{S}$ then sends SF to $\mathcal{C}$, who locally validates SF by deriving their own fk_s . Again, $\mathcal{C}$ is only able to check the SF because they have a full and complete view of the transcript.

In the regular (i.e two-party) variant of TLS 1.3 this approach has been shown to provide very high security guarantees: the pioneering work of [29] have shown that the TLS 1.3 handshake protocol establishes session keys with strong security properties under standard cryptographic assumptions. Interestingly, the TLS 1.3 handshake protocol actually achieves a stronger level of security than may be fully necessary in some settings. For example, it has been argued that the SF message alone authenticates the transcript [29], as the SF message is an authenticated HMAC of the transcript sent so far. Put differently, the SCV message does not provide any additional integrity guarantees for the

derived keying material compared to just checking the SF message.

TLS 1.3 in DECO and potential avenues for attack. The authors of [2] initially proposed using a variant of the above idea to accelerate certificate checking in DECO’s TLS 1.3 variant. Namely, the authors of DECO suggest, as the traffic keys are independent of the handshake keys, that the $\mathcal{C}$ and $\mathcal{V}$ can simply derive all secrets in a single circuit, with $\mathcal{C}$ alone learning all handshake keys for the purpose of certificate checking. Whilst it is unclear that this approach will work²⁰, we note that the description given in DECO does not specify whether $\mathcal{V}$ ever learns the SF or SCV message in the TLS 1.3 version of DECO, or if $\mathcal{C}$ alone has access to this information. We argue that this approach leads to a very straightforward attack. Namely, suppose that $\mathcal{C}$ is a malicious and wishes to falsely attest of data using a server $\mathcal{S}$. In this attack, $\mathcal{C}$ begins a three-party handshake with $\mathcal{V}$, and receives $\mathcal{V}$ ’s keyshare for the three-party handshake. Then, rather than contacting $\mathcal{S}$ and carrying out the usual three-party handshake, $\mathcal{C}$ simply replies to $\mathcal{V}$ with its own keyshare. Intuitively, this step is the same as establishing a TLS 1.3 session between $\mathcal{C}$ and $\mathcal{V}$ without the involvement of $\mathcal{S}$. This step produces the following outcomes:

- 1) $\mathcal{V}$, upon receipt of $\mathcal{C}$ ’s keyshare, is unable to determine whether it belongs to $\mathcal{S}$ or $\mathcal{C}$, as it is unauthenticated at this stage. Thus, $\mathcal{V}$ must assume that the keyshare is a legitimate value from $\mathcal{S}$.
- 2) On the other hand, $\mathcal{C}$ can now locally derive *all* handshake and traffic secrets for the session, as $\mathcal{C}$ knows the entirety of the shared handshake secret. Thus, $\mathcal{C}$ can systematically deceive $\mathcal{V}$ in an undetectable manner. Moreover, as $\mathcal{V}$ never learns the SF message (or even the SCV message), $\mathcal{V}$ is unable to check if the handshake transcript was valid. As such, if $\mathcal{C}$ indicates that the checks passed, then $\mathcal{V}$ continues as if the transcript was authentic.

At this stage, $\mathcal{C}$ can forge any TLS traffic between itself and $\mathcal{S}$, without $\mathcal{V}$ being able to detect the forgeries, leading to false attestations. Moreover, this attack persists even if $\mathcal{V}$ explicitly checks the SF message of the transcript: as the transcript between $\mathcal{V}$ and $\mathcal{C}$ is a valid TLS 1.3 transcript, the secret derivation process will produce valid secrets, and checks on SF will pass.

Formally speaking, this attack arises from the fact that, although $\mathcal{V}$ and $\mathcal{C}$ are treated as a singular entity from the perspective of $\mathcal{S}$, they are in fact very different entities. Practically speaking, the main issue is that $\mathcal{C}$ obtains a valid copy of the transcript, whereas $\mathcal{V}$ does not. Moreover, ensuring that $\mathcal{V}$ receives any meaningful authenticity guarantees in the face of a malicious adversary whilst respecting privacy is somewhat difficult. Indeed, a trivial solution to the aforementioned attack would be to simply require that $\mathcal{V}$ learns the identity of $\mathcal{S}$ and the SCV value, allowing $\mathcal{V}$ to authenticate $\mathcal{S}$.

²⁰. Given that deriving the traffic keys requires SF to be incorporated into the transcript hash, it seems impossible that the application and traffic secrets could be derived in one step.

Attack vectors on the Janus protocol. We argue that the recently proposed Janus [15] TLS attestation protocol is potentially vulnerable to an attack with a similar root cause, albeit with a different mechanism.

Briefly, the Janus protocol is a hybrid between DiStefano and DECO’s proxy mode. In its presented configuration, Janus treats the verifier as a TLS 1.3 proxy that forwards (and records) all traffic between $\mathcal{C}$ and $\mathcal{S}$. Notably, the Janus protocol differs from DiStefano by using a set of semi-honest garbled circuits for all secret derivation, other than AES-GCM tagging and verification. In this model, the adversary follows an agreed-upon protocol without deviation. Instead, the primary aim of the adversary is to learn the secret inputs of the honest party. Whilst this change is primarily made for efficiency, the authors of Janus also claim that the authenticity guarantees given by the SCV and SF messages allows for this change to be made while ensuring security against malicious adversaries.

In order to highlight a potential attack, we now briefly describe the differences in secret derivation between Janus and DiStefano. We note that the protocols are essentially identical up until the secret derivation procedure, and thus we omit these details.

- 1) $\mathcal{C}$ and $\mathcal{V}$ invoke a semi-honest garbled circuit protocol, that outputs the SHTS value to $\mathcal{C}$. $\mathcal{C}$ uses an authentic copy of the transcript to check the SF and SCV messages. If the checks fail, then $\mathcal{C}$ aborts. Otherwise, $\mathcal{C}$ reveals SHTS to $\mathcal{V}$, and $\mathcal{V}$ repeats the same checks.
- 2) $\mathcal{C}$ and $\mathcal{V}$ repeat similarly for the CHTS circuit.
- 3) Finally, $\mathcal{C}$ and $\mathcal{V}$ derive the traffic secrets using a series semi-honest garbled circuits. We note that, according to Figure 4 of [15], the dHS secret appears to be derived *after* the SHTS value has been checked. As we shall soon explain, this is problematic when only semi-honest garbled circuits are used.

We now present a potential attack using a malicious adversary that is undetectable using semi-honest garbled circuits. First, assume that $\mathcal{V}$ is a malicious verifier that wishes to compromise the confidentiality of $\mathcal{C}$. In order to achieve this, $\mathcal{V}$ generates a public key K_v and establishes a TLS session with $\mathcal{S}$, recording the entire transcript T and deriving all traffic secrets. Note that from the perspective of $\mathcal{S}$, $\mathcal{V}$ is behaving honestly, and thus $\mathcal{S}$ cannot mitigate this attack. Then, when $\mathcal{C}$ begins the Janus protocol, $\mathcal{V}$ replays the transcript T to $\mathcal{C}$ and maliciously garbles the SHTS derivation circuit such that the SHTS from T is revealed. We stress that this step requires $\mathcal{V}$ to be a malicious actor, as the circuit would otherwise output a different SHTS due to the keyshare input by $\mathcal{C}$. At this stage, $\mathcal{C}$ will check the SF and SCV messages and conclude that they are valid. $\mathcal{V}$, hence, has succeeded in their attack: $\mathcal{V}$ can simply ignore the request for validation using the CHTS, and, as they also know all other traffic secrets, $\mathcal{V}$ can undetectably decrypt or modify any messages exchanged between $\mathcal{S}$ and $\mathcal{C}$. Moreover, even if $\mathcal{V}$ is unable to modify the SHTS check, it seems clear that modifying the output of the dHS circuit to a known value is enough to remove any security guarantees that Janus offers;

indeed, setting the dHS to some known value affords the exact same powers to $\mathcal{V}$ in a manner that is undetectable to $\mathcal{C}$.

Attack intuition. All the aforementioned attacks rely on the mismatch between the transcript seen by $\mathcal{V}$ and the transcript seen by $\mathcal{C}$. On the one hand, omitting certain information from the view of $\mathcal{V}$ allows $\mathcal{C}$ to act in an arbitrarily powerful fashion, providing no guarantees at all to $\mathcal{C}$. Yet, we consider the attack on Janus to be equally as serious; here, the session guarantees expected by $\mathcal{C}$ simply fade away in an undetectable manner, even though $\mathcal{C}$ may have access to a full view of the transcript. In both cases, the attacks exist simply because there is no concrete binding between the transcript shown to $\mathcal{C}$ by $\mathcal{V}$ and the actual, underlying session that is being carried out.

Mitigating these attacks appears fairly straightforward. DiStefano avoids the attack on DECO’s TLS 1.3 variant by simply requiring that $\mathcal{C}$ provides commitments to session traffic as part of the authentication process. This, coupled with a ZKPVS scheme attesting to the validity of the SCV value, is enough to ensure that $\mathcal{C}$ cannot fool $\mathcal{V}$ without a collaborating $\mathcal{S}$. Moreover, the use of maliciously-secure garbled circuit protocols is enough to prevent $\mathcal{V}$ from modifying the output of the SHTS derivation in a predictable or useful fashion. Finally, we mention that a potential defence against the attack described on Janus would be to simply require that $\mathcal{V}$ proves that the value it sends in the client handshake somehow involves the share provided by $\mathcal{C}$. This could be achieved by revealing both $T = K_v + K_c$ and K_v to $\mathcal{C}$, which would allow $\mathcal{C}$ to ensure that its view of the transcript is authentic relative to the current session. However, we caution against the belief that this safeguard alone is sufficient, as it is clear that any attack against the derivation of dHS is sufficient to allow a malicious adversary to break the security of the Janus protocol. We leave clarifying the security properties of a DiStefano-like protocol that uses semi-honest primitives for future work.

Appendix J. Modifications in Implementation

We briefly list the changes introduced in our implementation. Recall that our implementation is over the BoringSSL library, so, in order to enhance client-side functionality, set and get functions have been developed for the key share structure of it, accompanied by thorough testing to ensure their reliability. Additionally, a “set_verifier” function has been introduced, allowing the client to establish the verifier structure, which is also supported by rigorous testing.

Furthermore, a new namespace, called “ThreeParty-Handshake”, has been created to facilitate communication with a third party in generating shared key shares. Moreover, a “key_store” field has been integrated into the SSL structure to securely store the local share of the three-party handshake keys, thus preserving the state of the original key shares. Utility functions have also been developed for extracting the not identifier (NID) and converting constructed binary

blocks (CBB) to elliptic curve (EC) points. These enhancements collectively contribute to a more robust and efficient implementation of the key share structure and three-party handshake protocol, facilitating secure communications in cryptographic applications.

Appendix K.

In this section we present an attack on an *interpretation* of the TLS 1.3 variant of DECO and the recently presented Janus [15] TLS attestation mechanism. Notably, this attack allows either a malicious client to produce false attestations, or for a malicious proxy to fully decrypt all traffic. We stress that our interpretation of DECO’s TLS 1.3 variant may be different from exactly what the authors intended, due to lack of specificity, and thus the attack presented may not be directly applicable. However, the attack presented here applies to any TLS attestation protocol that does not carefully handle the sharing (and disclosure) of traffic secrets.

The SF message. The attack presented in this section exploits a flaw in how both Janus and DECO’s TLS 1.3 variant handle session authentication between the client and the verifier. In order to explain this flaw, we briefly recall the TLS 1.3 server authentication mechanism used in TLS 1.3 (see [29] for a more thorough explanation).

First, assume that $\mathcal{S}$ and $\mathcal{C}$ are establishing a TLS 1.3 session; and that $\mathcal{S}$ and $\mathcal{C}$ have exchanged both the hello messages (ClientHello and CH) and derived authentic copies of the handshake secrets SHTS and CHTS. At this stage, $\mathcal{S}$ authenticates itself to $\mathcal{C}$ as follows:

- 1) To ensure authenticity, $\mathcal{S}$ sends its certificate SCRT to $\mathcal{C}$, followed by a signature on the hash of the session transcript (SCV). Upon receipt of these messages, $\mathcal{C}$ checks the certificate and uses the corresponding public key to verify SCV. Notably, $\mathcal{C}$ is able to check SCV because $\mathcal{C}$ has a full view of the transcript.
- 2) To ensure integrity, $\mathcal{S}$ uses SHTS to derive the *finished* key fk_s . $\mathcal{S}$ then computes the *server finished* (SF) message, by computing a HMAC on the hash of the session transcript using fk_s . $\mathcal{S}$ then sends SF to $\mathcal{C}$, who locally validates SF by deriving their own fk_s . Again, $\mathcal{C}$ is only able to check the SF because they have a full and complete view of the transcript.

In the regular (i.e two-party) variant of TLS 1.3 this approach has been shown to provide very high security guarantees: the pioneering work of [29] have shown that the TLS 1.3 handshake protocol establishes session keys with strong security properties under standard cryptographic assumptions. Interestingly, the TLS 1.3 handshake protocol actually achieves a stronger level of security than may be fully necessary in some settings. For example, it has been argued that the SF message alone authenticates the transcript [29], as the SF message is an authenticated HMAC of the transcript sent so far. Put differently, the SCV message does not provide any additional integrity guarantees for the derived keying material compared to just checking the SF message.

TLS 1.3 in DECO and potential avenues for attack. The authors of [2] initially proposed using a variant of the above idea to accelerate certificate checking in DECO’s TLS 1.3 variant. Namely, the authors of DECO suggest, as the traffic keys are independent of the handshake keys, that the $\mathcal{C}$ and $\mathcal{V}$ can simply derive all secrets in a single circuit, with $\mathcal{C}$ alone learning all handshake keys for the purpose of certificate checking. Whilst it is unclear that this approach will work²¹, we note that the description given in DECO does not specify whether $\mathcal{V}$ ever learns the SF or SCV message in the TLS 1.3 version of DECO, or if $\mathcal{C}$ alone has access to this information. We argue that this approach leads to a very straightforward attack. Namely, suppose that $\mathcal{C}$ is a malicious and wishes to falsely attest of data using a server $\mathcal{S}$. In this attack, $\mathcal{C}$ begins a three-party handshake with $\mathcal{V}$, and receives $\mathcal{V}$ ’s keyshare for the three-party handshake. Then, rather than contacting $\mathcal{S}$ and carrying out the usual three-party handshake, $\mathcal{C}$ simply replies to $\mathcal{V}$ with its own keyshare. Intuitively, this step is the same as establishing a TLS 1.3 session between $\mathcal{C}$ and $\mathcal{V}$ without the involvement of $\mathcal{S}$. This step produces the following outcomes:

- 1) $\mathcal{V}$, upon receipt of $\mathcal{C}$ ’s keyshare, is unable to determine whether it belongs to $\mathcal{S}$ or $\mathcal{C}$, as it is unauthenticated at this stage. Thus, $\mathcal{V}$ must assume that the keyshare is a legitimate value from $\mathcal{S}$.
- 2) On the other hand, $\mathcal{C}$ can now locally derive *all* handshake and traffic secrets for the session, as $\mathcal{C}$ knows the entirety of the shared handshake secret. Thus, $\mathcal{C}$ can systematically deceive $\mathcal{V}$ in an undetectable manner. Moreover, as $\mathcal{V}$ never learns the SF message (or even the SCV message), $\mathcal{V}$ is unable to check if the handshake transcript was valid. As such, if $\mathcal{C}$ indicates that the checks passed, then $\mathcal{V}$ continues as if the transcript was authentic.

At this stage, $\mathcal{C}$ can forge any TLS traffic between itself and $\mathcal{S}$, without $\mathcal{V}$ being able to detect the forgeries, leading to false attestations. Moreover, this attack persists even if $\mathcal{V}$ explicitly checks the SF message of the transcript: as the transcript between $\mathcal{V}$ and $\mathcal{C}$ is a valid TLS 1.3 transcript, the secret derivation process will produce valid secrets, and checks on SF will pass.

Formally speaking, this attack arises from the fact that, although $\mathcal{V}$ and $\mathcal{C}$ are treated as a singular entity from the perspective of $\mathcal{S}$, they are in fact very different entities. Practically speaking, the main issue is that $\mathcal{C}$ obtains a valid copy of the transcript, whereas $\mathcal{V}$ does not. Moreover, ensuring that $\mathcal{V}$ receives any meaningful authenticity guarantees in the face of a malicious adversary whilst respecting privacy is somewhat difficult. Indeed, a trivial solution to the aforementioned attack would be to simply require that $\mathcal{V}$ learns the identity of $\mathcal{S}$ and the SCV value, allowing $\mathcal{V}$ to authenticate $\mathcal{S}$.

Attack vectors on the Janus protocol. We argue that the recently proposed Janus [15] TLS attestation protocol is

21. Given that deriving the traffic keys requires SF to be incorporated into the transcript hash, it seems impossible that the application and traffic secrets could be derived in one step.

potentially vulnerable to an attack with a similar root cause, albeit with a different mechanism.

Briefly, the Janus protocol is a hybrid between DiStefano and DECO’s proxy mode. In its presented configuration, Janus treats the verifier as a TLS 1.3 proxy that forwards (and records) all traffic between $\mathcal{C}$ and $\mathcal{S}$. Notably, the Janus protocol differs from DiStefano by using a set of semi-honest garbled circuits for all secret derivation, other than AES-GCM tagging and verification. In this model, the adversary follows an agreed-upon protocol without deviation. Instead, the primary aim of the adversary is to learn the secret inputs of the honest party. Whilst this change is primarily made for efficiency, the authors of Janus also claim that the authenticity guarantees given by the SCV and SF messages allows for this change to be made while ensuring security against malicious adversaries.

In order to highlight a potential attack, we now briefly describe the differences in secret derivation between Janus and DiStefano. We note that the protocols are essentially identical up until the secret derivation procedure, and thus we omit these details.

- 1) $\mathcal{C}$ and $\mathcal{V}$ invoke a semi-honest garbled circuit protocol, that outputs the SHTS value to $\mathcal{C}$. $\mathcal{C}$ uses an authentic copy of the transcript to check the SF and SCV messages. If the checks fail, then $\mathcal{C}$ aborts. Otherwise, $\mathcal{C}$ reveals SHTS to $\mathcal{V}$, and $\mathcal{V}$ repeats the same checks.
- 2) $\mathcal{C}$ and $\mathcal{V}$ repeat similarly for the CHTS circuit.
- 3) Finally, $\mathcal{C}$ and $\mathcal{V}$ derive the traffic secrets using a series semi-honest garbled circuits. We note that, according to Figure 4 of [15], the dHS secret appears to be derived *after* the SHTS value has been checked. As we shall soon explain, this is problematic when only semi-honest garbled circuits are used.

We now present a potential attack using a malicious adversary that is undetectable using semi-honest garbled circuits. First, assume that $\mathcal{V}$ is a malicious verifier that wishes to compromise the confidentiality of $\mathcal{C}$. In order to achieve this, $\mathcal{V}$ generates a public key K_v and establishes a TLS session with $\mathcal{S}$, recording the entire transcript T and deriving all traffic secrets. Note that from the perspective of $\mathcal{S}$, $\mathcal{V}$ is behaving honestly, and thus $\mathcal{S}$ cannot mitigate this attack. Then, when $\mathcal{C}$ begins the Janus protocol, $\mathcal{V}$ replays the transcript T to $\mathcal{C}$ and maliciously garbles the SHTS derivation circuit such that the SHTS from T is revealed. We stress that this step requires $\mathcal{V}$ to be a malicious actor, as the circuit would otherwise output a different SHTS due to the keyshare input by $\mathcal{C}$. At this stage, $\mathcal{C}$ will check the SF and SCV messages and conclude that they are valid. $\mathcal{V}$, hence, has succeeded in their attack: $\mathcal{V}$ can simply ignore the request for validation using the CHTS, and, as they also know all other traffic secrets, $\mathcal{V}$ can undetectably decrypt or modify any messages exchanged between $\mathcal{S}$ and $\mathcal{C}$. Moreover, even if $\mathcal{V}$ is unable to modify the SHTS check, it seems clear that modifying the output of the dHS circuit to a known value is enough to remove any security guarantees that Janus offers; indeed, setting the dHS to some known value affords the exact same powers to $\mathcal{V}$ in a manner that is undetectable to

$\mathcal{C}$.

Attack intuition. All the aforementioned attacks rely on the mismatch between the transcript seen by $\mathcal{V}$ and the transcript seen by $\mathcal{C}$. On the one hand, omitting certain information from the view of $\mathcal{V}$ allows $\mathcal{C}$ to act in an arbitrarily powerful fashion, providing no guarantees at all to $\mathcal{C}$. Yet, we consider the attack on Janus to be equally as serious; here, the session guarantees expected by $\mathcal{C}$ simply fade away in an undetectable manner, even though $\mathcal{C}$ may have access to a full view of the transcript. In both cases, the attacks exist simply because there is no concrete binding between the transcript shown to $\mathcal{C}$ by $\mathcal{V}$ and the actual, underlying session that is being carried out.

Mitigating these attacks appears fairly straightforward. DiStefano avoids the attack on DECO’s TLS 1.3 variant by simply requiring that $\mathcal{C}$ provides commitments to session traffic as part of the authentication process. This, coupled with a ZKPVS scheme attesting to the validity of the SCV value, is enough to ensure that $\mathcal{C}$ cannot fool $\mathcal{V}$ without a collaborating $\mathcal{S}$. Moreover, the use of maliciously-secure garbled circuit protocols is enough to prevent $\mathcal{V}$ from modifying the output of the SHTS derivation in a predictable or useful fashion. Finally, we mention that a potential defence against the attack described on Janus would be to simply require that $\mathcal{V}$ proves that the value it sends in the client handshake somehow involves the share provided by $\mathcal{C}$. This could be achieved by revealing both $T = K_v + K_c$ and K_v to $\mathcal{C}$, which would allow $\mathcal{C}$ to ensure that its view of the transcript is authentic relative to the current session. However, we caution against the belief that this safeguard alone is sufficient, as it is clear that any attack against the derivation of dHS is sufficient to allow a malicious adversary to break the security of the Janus protocol. We leave clarifying the security properties of a DiStefano-like protocol that uses semi-honest primitives for future work.

Appendix L. Glossary of Terms

We provide a list of common acronyms used below. Note that acronyms used in the specification of DiStefano in Fig. 2 are respectively defined in Appendix A.

2PC. Two-party computation.

AD. Associated Data.

AES. Advanced Encryption Standard.

AES-GCM. AES in Galois Counter Mode.

CBC. Cipher Block Chaining

CP. Commitment phase.

DCTLS. Designated-Commitment TLS.

ECDH. Elliptic Curve Diffie-Hellman.

ECtF. Elliptic Curve to Field operation.

HSP. Handshake phase.

MPC. Multi-party computation.

MtA. Multiplicative to Affine operation.

OT. Oblivious Transfer.
PRF. Pseudo-Random Function.
QP. Query phase.
RL. Record-layer protocol.
TLS. Transport Layer Security protocol.
ZKPoK. Zero Knowledge Proof of Knowledge.
ZKPVS. Zero Knowledge Proof of Valid Signature.